গত ২০ বছর ধরে সিস্টেম অ্যানালাইসিস অ্যান্ড ডিজাইন (System Analysis and Design) পড়াতে গিয়ে আমি দেখেছি, যেকোনো কঠিন বিষয় একটি সুন্দর ও বাস্তব গল্পের মাধ্যমে শিখলে তা সারা জীবন মনে থাকে।

আজ আমি তোমাদের '**Sources and Types of Information**' বা **তথ্যের উৎস এবং প্রকারভেদ** সম্পর্কে জানাবো একটি অত্যন্ত পরিচিত গল্পের মাধ্যমে।

গল্প: রহিম সাহেবের 'বেঙ্গল মার্ট'

ধরে নাও, রহিম সাহেবের 'বেঙ্গল মার্ট' নামে একটি জনপ্রিয় সুপারশপ আছে। ব্যবসা যখন ছোট ছিল, তখন রহিম সাহেব একাই সব সিদ্ধান্ত নিতেন। কিন্তু ব্যবসা বড় হওয়ার সাথে সাথে তার সঠিক সিদ্ধান্ত নেওয়ার জন্য বিভিন্ন ধরনের তথ্য বা **Information**-এর প্রয়োজন হলো। এই Information হলো মূলত এমন প্রক্রিয়াজাত ডাটা (Processed data) যা প্রতিষ্ঠানের ম্যানেজারদের কার্যকরী সিদ্ধান্ত নিতে এবং দক্ষতার সাথে ব্যবসা পরিচালনা করতে সাহায্য করে।

রহিম সাহেবের কোম্পানিতে মূলত তিন স্তরের মানুষ কাজ করেন এবং তাদের তথ্যের প্রয়োজনও তিন রকম। রহিম সাহেব হলেন কোম্পানির প্রধান (Top Manager)। তিনি আগামী ৫ বছরে শহরে আরও ৫টি নতুন শাখা খোলার কথা ভাবছেন। এই দীর্ঘমেয়াদী পরিকল্পনার জন্য তার যে তথ্যের প্রয়োজন, তাকে বলা হয় **Strategic Information** (স্ট্র্যাটেজিক ইনফরমেশন)।

তার নিচে কাজ করেন ম্যানেজার করিম (Middle Manager)। করিমের দায়িত্ব হলো আগামী ঈদে ক্রেতাদের কী ধরনের ডিসকাউন্ট দেওয়া হবে বা কোন সাপ্লায়ারের কাছ থেকে কত টাকার পণ্য কেনা হবে তা ঠিক করা। এই স্বল্পমেয়াদী এবং কৌশলগত পরিকল্পনার জন্য করিম যে তথ্য ব্যবহার করেন, তা হলো **Tactical Information** (ট্যাকটিকাল ইনফরমেশন)।

আর একদম নিচে কাজ করেন সুপারশপের ক্যাশিয়ার এবং ইনভেন্টরি স্টাফরা (Line Managers/Operational level)। প্রতিদিন কতগুলো সাবান বা চাল বিক্রি হলো, কোন পণ্যটি দ্রুত পুনরায় অর্ডার করতে হবে—এই দৈনন্দিন কাজের জন্য তাদের লাগে **Operational Information** (অপারেশনাল ইনফরমেশন)। এর বাইরেও, মাস শেষে সরকারকে কত ট্যাক্স দিতে হবে, সেই আইনগত হিসাবের জন্য কোম্পানির যে তথ্য লাগে তাকে বলা হয় **Statutory Information** (স্ট্যাটুটরি ইনফরমেশন)।

এখন প্রশ্ন হলো, রহিম সাহেব ও তার কর্মীরা এই সিদ্ধান্ত নেওয়ার তথ্যগুলো পান কোথা থেকে? অর্থাৎ, **Sources of Information** (তথ্যের উৎস) কী?

প্রতিষ্ঠানের ভেতরের কাজ থেকে যে তথ্য তৈরি হয়, যেমন—গত মাসের লাভের হিসাব বা ক্যাশ ফ্লো, তাকে বলা হয় **Internal information** (ইন্টারনাল ইনফরমেশন)। আর বাইরের পরিবেশ বা প্রতিযোগী কোম্পানি কী করছে, সরকারি নতুন আইন কী—এগুলো হলো **External information** (এক্সটারনাল ইনফরমেশন)।

এছাড়াও তথ্য সংগ্রহের পদ্ধতি অনুযায়ী মূল তিনটি উৎস রয়েছে। যখন 'বেঙ্গল মার্ট'-এ কোনো ক্রেতা সরাসরি একটি ইনভয়েস (Invoice) বা রশিদ পায়, তখন সেই অরিজিনাল ডকুমেন্টটিকে বলা হয় **Primary Source** (প্রাইমারি সোর্স)। করিম যখন মাস শেষে এই প্রাইমারি ইনভয়েসগুলোর হিসাব যাচাই করে একটি ব্যাংক স্টেটমেন্ট বিশ্লেষণ করেন, তখন সেই ব্যাখ্যা বা বিশ্লেষণমূলক তথ্যটি হয়ে যায় **Secondary Source** (সেকেন্ডারি সোর্স)। আর যখন রহিম সাহেব তার ব্যবসার পরিধি বাড়ানোর জন্য ডিরেক্টরি (Directories) বা উইকিপিডিয়া (Wikipedia) ঘেঁটে নতুন সাপ্লায়ারদের তথ্য খোঁজেন, তখন সেই সংকলিত তথ্যভাণ্ডারকে বলা হয় **Tertiary Source** (টারশিয়ারি সোর্স)।

এভাবেই সঠিক উৎস থেকে সঠিক প্রকারের তথ্য সংগ্রহ করে রহিম সাহেবের 'বেঙ্গল মার্ট' আজ একটি সফল প্রতিষ্ঠানে পরিণত হয়েছে।

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

Types of Information (তথ্যের প্রকারভেদ):

- **Strategic Information (স্ট্র্যাটেজিক ইনফরমেশন):** এটি দীর্ঘমেয়াদী পরিকল্পনার (Long-range planning) জন্য ব্যবহৃত হয় এবং এটি তুলনামূলক কম স্ট্রাকচারড (Unstructured) হয়। টপ ম্যানেজাররা (Top Managers) নতুন শাখা খোলা বা ব্যবসার প্রসার নিয়ে সিদ্ধান্ত নিতে এটি ব্যবহার করেন।
- **Tactical Information (ট্যাকটিকাল ইনফরমেশন):** মিডল লেভেল ম্যানেজাররা (Middle Managers) স্বল্পমেয়াদী সিদ্ধান্ত (Short-range improvement) এবং প্রজেক্ট নিয়ন্ত্রণের জন্য এটি ব্যবহার করেন।
- **Operational Information (অপারেশনাল ইনফরমেশন):** প্রতিদিনের কাজ (Day-to-day operations) পরিচালনার জন্য লাইন ম্যানেজাররা (Line managers) এটি ব্যবহার করেন, যেমন—দৈনিক বিক্রি বা বিলিং।
- **Statutory Information (স্ট্যাটুটরি ইনফরমেশন):** আইনগত কারণে সরকারি কর্তৃপক্ষের কাছে জমা দেওয়ার জন্য এই তথ্য প্রয়োজন হয়, যেমন—সেলস ট্যাক্স রিটার্ন (Sales tax return)

Sources of Information (তথ্যের উৎস):

- **Internal Information (ইন্টারনাল ইনফরমেশন):** প্রতিষ্ঠানের ভেতরের অপারেশন থেকে উৎপন্ন তথ্য, যেমন—মার্কেটিং, সেলস, লাভ বা খরচের হিসাব।
- **External Information (এক্সটারনাল ইনফরমেশন):** প্রতিষ্ঠানের বাইরের পরিবেশ থেকে প্রাপ্ত তথ্য, যেমন—সরকারি সংস্থা বা প্রাইভেট ডিরেক্টরির তথ্য।
- **Primary Sources (প্রাইমারি সোর্স):** এটি হলো একদম অরিজিনাল বা প্রথম ডাটা, যার কোনো পরিবর্তন বা বিশ্লেষণ করা হয়নি। যেমন—অরিজিনাল ইনভয়েস (invoice), চেক (cheque) বা থিসিস (thesis)
- **Secondary Sources (সেকেন্ডারি সোর্স):** প্রাইমারি সোর্সের বিশ্লেষণ বা প্রক্রিয়াজাত রূপ, যেমন—বই, সম্পাদিত কাজ, বা ব্যাংক স্টেটমেন্ট যা প্রাইমারি চেকগুলোর হিসাব দেয়।
- **Tertiary Sources (টারশিয়ারি সোর্স):** অন্যান্য সোর্সগুলোকে যখন এক জায়গায় সাজানো বা সংকলিত করা হয়, যেমন—উইকিপিডিয়া (Wikipedia), ডিকশনারি বা ডিরেক্টরি।

আশা করি এই গল্পের মাধ্যমে System Analysis and Design-এর এই গুরুত্বপূর্ণ বিষয়টি তোমাদের কাছে একেবারে পরিষ্কার হয়েছে!

আজ আমি তোমাদের **Information Management System** বা আরও নির্দিষ্টভাবে বলতে গেলে **Management Information System (MIS)** এবং এর আশেপাশের সিস্টেমগুলোর ধারণা দেব একটি খুব সুন্দর ও পরিচিত গল্পের মাধ্যমে।

গল্প: রহিম সাহেবের "ডিজিটাল ইলেকট্রনিক্স"

ধরে নাও, রহিম সাহেবের শহরের তিন জায়গায় 'ডিজিটাল ইলেকট্রনিক্স' নামের তিনটি বড় শোরুম আছে। শুরুতে ব্যবসা ছোট ছিল, কিন্তু এখন প্রতিদিন শত শত ল্যাপটপ, ফ্রিজ, টিভি বিক্রি হয়।

রহিম সাহেবের শোরুমে তিন স্তরের কর্মী কাজ করেন। একদম নিচে আছেন ক্যাশিয়ার ও সেলসম্যানরা (Operational level)। প্রতিদিন তারা কাস্টমারদের বিল করেন এবং কোন পণ্য বিক্রি হলো তার হিসাব রাখেন। এই যে প্রতিদিনের রুটিন কাজ এবং কেনাবেচার হিসাব রাখা—এটার জন্য তারা যে সিস্টেমটি ব্যবহার করেন, তাকে বলা হয় **Transaction Processing System (TPS)**। এর কাজ হলো শুধু দৈনন্দিন কাঁচা তথ্য বা **Data** সংগ্রহ করা।

কিন্তু সমস্যা শুরু হলো অন্য জায়গায়।

রহিম সাহেবের মিডল ম্যানেজার (Middle Manager) হলেন করিম। মাস শেষে করিম সাহেবের পাগল হওয়ার উপক্রম হয়! রহিম সাহেব জানতে চান—কোন শোরুমে গত মাসে লাভ বেশি হয়েছে? কোন ব্যান্ডের ফ্রিজগুলো স্টকে পড়ে আছে এবং আর কেনা উচিত নয়? করিম সাহেবকে হাজার হাজার বিলের কাগজ ঘেঁটে এই প্রশ্নের উত্তর বের করতে হয়। কাঁচা **Data** থেকে সিদ্ধান্ত নেওয়ার মতো **Information** বা তথ্য বের করাটা তার জন্য দুঃসাধ্য হয়ে দাঁড়ায়।

একদিন আমি রহিম সাহেবের শোরুমে গেলাম। করিম সাহেবের দুরবস্থা দেখে আমি তাদের একটি **Management Information System (MIS)** বা ইনফরমেশন ম্যানেজমেন্ট সিস্টেম সেটআপ করার পরামর্শ দিলাম।

আমি করিম সাহেবকে বুঝিয়ে বললাম, "দেখুন, আপনার ক্যাশিয়াররা প্রতিদিন যে বিল বানাচ্ছে (TPS), এই MIS সেই ডাটাগুলোকে স্বয়ংক্রিয়ভাবে সংগ্রহ করবে। এরপর এটি সেই ডাটা প্রসেস করে মাস শেষে আপনার সামনে সুন্দর একটি সামারি রিপোর্ট (Summary Report) বা গ্রাফ তৈরি করে দেবে। আপনি এক ক্লিকেই দেখতে পাবেন কোন শোরুমে সেলস কেমন, আর কোন প্রোডাক্ট দ্রুত শেষ হচ্ছে।"

MIS মূলত মিডল ম্যানেজারদের দৈনন্দিন পরিকল্পনা এবং নিয়ন্ত্রণের (Planning and controlling) কাজে সাহায্য করে।

এই MIS বসানোর পর করিম সাহেবের কাজ খুব সহজ হয়ে গেল। কিন্তু রহিম সাহেব হলেন কোম্পানির মালিক (Top Executive)। তিনি আগামী ৫ বছরে পুরো দেশে আরও ১০টি শোরুম দিতে চান। তার দরকার আরও বড় পরিসরের ধারণা। তখন আমি তাকে পরিচয় করালাম **Executive Support System (ESS)** এর সাথে, যা তাকে একটি ডিজিটাল ড্যাশবোর্ডে (Digital Dashboard) পুরো কোম্পানির লাভ-ক্ষতি ও ভবিষ্যতের কৌশলগত সিদ্ধান্ত (Strategic decisions) নিতে সাহায্য করবে।

আর যদি কখনো রহিম সাহেব বা করিম সাহেবকে এমন কোনো আকস্মিক বা জটিল সিদ্ধান্ত নিতে হয়, যার কোনো নির্দিষ্ট নিয়ম নেই (Unstructured decisions)—যেমন, হঠাৎ করোনা মহামারীর কারণে সাপ্লাই চেইন বন্ধ হয়ে গেলে কী করা উচিত, তখন তারা যে অ্যানালিটিক্যাল সিস্টেমের সাহায্য নেবেন, তাকে বলা হয় **Decision Support System (DSS)**।

গল্পের মূল কথা হলো: প্রতিদিনের কাজের ডাটা সংগ্রহ করে TPS, সেই ডাটাকে প্রসেস করে ম্যানেজারদের জন্য সুন্দর রিপোর্ট তৈরি করে দেয় MIS, কঠিন সিদ্ধান্ত নিতে সাহায্য করে DSS, আর মালিকদের দীর্ঘমেয়াদী লক্ষ্য পূরণে কাজ করে ESS! সব মিলিয়ে এভাবেই একটি প্রতিষ্ঠানের **Information System** সফলভাবে কাজ করে।

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

- **Information System (ইনফরমেশন সিস্টেম):** এটি হলো একটি প্রাতিষ্ঠানিক ব্যবস্থা যা তথ্য (Information) সংগ্রহ, প্রক্রিয়াজাত (Process), সংরক্ষণ (Store) এবং বিতরণ (Distribute) করার জন্য ডিজাইন করা হয়।
- **Management Information System (MIS):**
    - এটি মূলত মিডল ম্যানেজারদের (Middle managers) জন্য তৈরি একটি সিস্টেম।
    - এটি **Transaction Processing System (TPS)** থেকে প্রাপ্ত দৈনন্দিন ডাটা ব্যবহার করে।
    - এটি প্রতিষ্ঠানের বর্তমান পারফরম্যান্স পর্যবেক্ষণ, নিয়ন্ত্রণ (monitor and control) এবং ভবিষ্যতের পারফরম্যান্স সম্পর্কে পূর্বাভাস দিতে সাহায্য করে।
    - এটি ম্যানেজারদের রুটিনমাফিক সিদ্ধান্ত নিতে সামারাইজড রিপোর্ট (Summarized reports) প্রদান করে (যেমন- সেলস রিপোর্ট, ইনভেন্টরি রিপোর্ট)।
- **Transaction Processing System (TPS):** এটি দৈনন্দিন ব্যবসায়িক লেনদেন (Day-to-day business transactions) পরিচালনা করে। যেমন- বিলিং বা পে-রোল।

- **Decision Support System (DSS):** এটি নন-রুটিন (Non-routine) বা আনস্ট্রাকচারড (Unstructured) সিদ্ধান্ত নেওয়ার ক্ষেত্রে অ্যানালিটিক্যাল টুলস বা ডাটা অ্যানালাইসিসের মাধ্যমে ম্যানেজারদের সাহায্য করে। এটি অভ্যন্তরীণ ডাটার পাশাপাশি বাইরের ডাটাও (যেমন- প্রতিযোগী কোম্পানির প্রোডাক্টের দাম) ব্যবহার করে।
- **Executive Support System (ESS):** এটি টপ ম্যানেজমেন্ট বা এক্সিকিউটিভদের (Top-level executives) দীর্ঘমেয়াদী ও কৌশলগত (Strategic) সিদ্ধান্ত নিতে সাহায্য করে। এটি সাধারণত গ্রাফ বা ডিজিটাল ড্যাশবোর্ড (Digital dashboards) আকারে তথ্য উপস্থাপন করে।

আশা করি এই বাস্তব গল্পের মাধ্যমে বিভিন্ন ধরনের Information Management System-এর ধারণা এবং তাদের কাজ তোমার কাছে একেবারে জলের মতো পরিষ্কার হয়ে গেছে!

আজ আমি তোমাদের একজন সিস্টেম অ্যানালিস্টের ভূমিকা এবং দায়িত্বগুলো (**Roles and Responsibilities**) একটি চমৎকার গল্পের মাধ্যমে বোঝাবো, ঠিক যেভাবে আগের ক্লাসে আমরা ইনফরমেশন সিস্টেম শিখেছিলাম।

গল্প: হাসান সাহেবের 'বইঘর' এবং একজন তারিক

ধরে নাও, হাসান সাহেবের 'বইঘর' নামে দেশজুড়ে একটি বিশাল বইয়ের রিটেইল চেইন আছে। কিন্তু সম্প্রতি তার ব্যবসায় চরম বিশৃঙ্খলা দেখা দিয়েছে। ক্রেতারা অনলাইনে বই অর্ডার করে পাচ্ছেন না, ইনভেন্টরিতে বইয়ের স্টক মিলছে না, আর কর্মচারীরা সারাদিন খাতা-কলম আর পুরোনো সফটওয়্যার নিয়ে হিমশিম খাচ্ছে।

এই সমস্যা সমাধানের জন্য হাসান সাহেব তারিক নামের একজন অভিজ্ঞ **System Analyst**-কে নিয়োগ দিলেন। তারিকের কাজ শুধু একটি সফটওয়্যার বানিয়ে দেওয়া নয়; তার কাজ অনেক বিস্তৃত। চলো দেখি তারিক কীভাবে কাজ করে:

প্রথমেই তারিক একজন **Investigator (অনুসন্ধানকারী)** এবং **Consultant (পরামর্শদাতা)** হিসেবে কাজ শুরু করল। সে সোজা কম্পিউটারে বসে কোডিং শুরু করল না। সে চলে গেল গোডাউনে, কথা বলল সেলসম্যানদের সাথে, এবং ম্যানেজমেন্টের সাথে মিটিং করে বোঝার চেষ্টা করল আসলে সমস্যাটা কোথায়। সে দেখল, তাদের বর্তমান সিস্টেমে কোনো স্বয়ংক্রিয় আপডেট হয় না।

এরপর তারিককে হতে হলো একজন অনুবাদক! অর্থাৎ, সে ব্যবসার সমস্যাগুলোকে টেকনোলজির ভাষায় রূপান্তর করল। তার এই দায়িত্বটিকে বলা হয় **Translating business needs into IT solutions**। হাসান সাহেবের ব্যবসায়িক চাহিদাগুলো পূরণ করার জন্য তারিক একটি নতুন সেন্ট্রাল ডাটাবেস এবং ই-কমার্স প্ল্যাটফর্মের নকশা তৈরি করল। এই পর্যায়ে সে একজন **Architect (স্থপতি)** হিসেবে সিস্টেমের আর্কিটেকচার, ডাটাবেস এবং ইন্টারফেস কেমন হবে তা ডিজাইন করল।

কিন্তু শুধু ভালো সিস্টেম বানালেই তো হবে না, মানুষ তো সেটা ব্যবহার করতে চাইবে না! বইঘরের অনেক পুরোনো কর্মচারী কম্পিউটারের নতুন সিস্টেম দেখে ভয় পেয়ে গেল, তারা ভাবল তাদের চাকরি চলে যাবে। এই পর্যায়ে তারিককে অবতীর্ণ হতে হলো একজন **Psychologist (মনোবিজ্ঞানী)** এবং **Motivator (অনুপ্রেরণাদাতা)**-এর ভূমিকায়। সে তাদের ভয় দূর করল, তাদের সাইকোলজি বুঝে সিস্টেমটিকে অত্যন্ত সহজ (user-friendly) করে ডিজাইন করল, এবং তাদের বোঝাল যে এই সিস্টেম তাদের কাজকে কত সহজ করে দেবে।

সিস্টেমটি যখন তৈরি হতে লাগল, তারিক তখন প্রোগ্রামার (ডেভেলপার), ম্যানেজমেন্ট এবং সাধারণ ইউজারদের মাঝে একটি সেতুবন্ধন হিসেবে কাজ করল। তার এই দায়িত্বটি হলো **Coordinating with stakeholders (স্টেকহোল্ডারদের সাথে সমন্বয় করা)**।

অবশেষে নতুন সিস্টেম চালু হলো। তারিক একজন **Supporting Expert (সহায়ক বিশেষজ্ঞ)** হিসেবে কর্মীদের ট্রেনিং দিল (Training users) এবং সিস্টেমটি যাতে ঠিকঠাক চলে তা নিশ্চিত করল। তারিকের এই পুরো যাত্রায়, সে কেবল একটি সফটওয়্যারই বদলায়নি, সে পুরো 'বইঘর'-এর কাজ করার ধরনে একটি ইতিবাচক পরিবর্তন এনেছে। তাই তাকে বলা যায় একজন **Agent of Change (পরিবর্তনের প্রতিনিধি)**।

তারিকের এই গল্পের মতোই, বাস্তব জীবনে একজন সিস্টেম অ্যানালিস্টকে ব্যবসা এবং প্রযুক্তির মধ্যে সেতুবন্ধন তৈরি করতে হয়।

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

গল্পের আলোকে একজন **System Analyst**-এর প্রধান ভূমিকা ও দায়িত্বসমূহ (Roles and Responsibilities) নিচে বুলেট পয়েন্টে দেওয়া হলো:

- **Investigator and Problem Solver (অনুসন্ধানকারী এবং সমস্যা সমাধানকারী):** বর্তমান সিস্টেম বা ব্যবসার সমস্যাগুলো গভীরভাবে অনুসন্ধান করা এবং সেগুলো সমাধানের জন্য সঠিক প্রযুক্তিগত পথ বের করা।
- **Translating Business Needs into IT Solutions (ব্যবসায়িক চাহিদাকে আইটি সলিউশনে রূপান্তর):** সাধারণ ইউজার এবং ম্যানেজমেন্টের ব্যবসায়িক লক্ষ্যগুলোকে (Business requirements) বুঝে সেগুলোকে সফটওয়্যার বা আইটি সমাধানে রূপান্তর করা।
- **Consultant and Supporting Expert (পরামর্শদাতা এবং সহায়ক বিশেষজ্ঞ):** প্রতিষ্ঠানের বাইরে থেকে বা ভেতর থেকে একজন বিশেষজ্ঞ হিসেবে সিস্টেমের হার্ডওয়্যার, সফটওয়্যার এবং কাজের প্রক্রিয়া নিয়ে সঠিক পরামর্শ দেওয়া এবং ইউজারদের গাইড করা।
- **Agent of Change (পরিবর্তনের প্রতিনিধি):** একটি নতুন সিস্টেম আনার মাধ্যমে প্রতিষ্ঠানের কাজের পরিবেশে ইতিবাচক পরিবর্তন আনা এবং পরিবর্তনের একটি সুস্পষ্ট পরিকল্পনা তৈরি করা।
- **Architect and Designer (স্থপতি এবং নকশাকার):** নতুন সিস্টেমের আর্কিটেকচার, ডাটাবেস, ইনপুট/আউটপুট এবং ইউজার ইন্টারফেস ডিজাইন করা।

- **Coordinator with Stakeholders (সমন্বয়কারী):** সিস্টেম ডেভেলপার, প্রোগ্রামার, ইউজার, ম্যানেজমেন্ট এবং ভেন্ডরদের (Vendors) মধ্যে সফলভাবে যোগাযোগ ও সমন্বয় রক্ষা করা।
- **Psychologist and Motivator (মনোবিজ্ঞানী এবং অনুপ্রেরণাদাতা):** ইউজারদের প্রযুক্তিভীতি ও মানসিকতা বোঝা এবং নতুন সিস্টেম গ্রহণে তাদের অনুপ্রাণিত করা।
- **Project Planning & Training (প্রজেক্ট প্ল্যানিং এবং প্রশিক্ষণ):** প্রজেক্টের সময়সূচি ও খরচ নির্ধারণ করা, প্রসেসগুলো ডকুমেন্ট করা এবং নতুন সিস্টেম সঠিকভাবে ব্যবহারের জন্য ইউজারদের প্রশিক্ষণ (Train users) দেওয়া।

আশা করি এই গল্পের মাধ্যমে একজন System Analyst-এর কাজ আসলে কতটা বৈচিত্র্যময় ও গুরুত্বপূর্ণ, তা তোমার কাছে স্পষ্ট হয়েছে!

আবার ফিরে এলাম তোমাদের সিস্টেম অ্যানালাইসিস অ্যান্ড ডিজাইনের (System Analysis and Design) ক্লাসে। গত ক্লাসে আমরা দেখেছি একজন সিস্টেম অ্যানালিস্ট প্রতিষ্ঠানের রূপকার হিসেবে কী কী দায়িত্ব পালন করেন। আজ আমরা জানবো, সেই কাজগুলো নিখুঁতভাবে করার জন্য তার ভেতরে কী কী গুণ বা **Qualities** থাকতে হয়। চলো, একটি নতুন গল্পের মাধ্যমে বিষয়টি বুঝে নিই।

গল্প: 'কুইকড্রপ লজিস্টিকস' এবং ফারহানের জাদুকরী সমাধান

ধরে নাও, 'কুইকড্রপ লজিস্টিকস' (QuickDrop Logistics) নামে একটি খুব জনপ্রিয় ডেলিভারি কোম্পানি রয়েছে। সম্প্রতি তাদের ব্যবসা হঠাৎ খুব বেড়ে যায়, কিন্তু এর সাথে পাল্লা দিয়ে শুরু হয় বিশৃঙ্খলা। প্রতিদিন হাজার হাজার পার্সেল ভুল ঠিকানায় চলে যাচ্ছিল, ডেলিভারি ম্যানরা বিভ্রান্ত হচ্ছিল এবং কাস্টমাররা রেগে যাচ্ছিল। কোম্পানির মালিক বাধ্য হয়ে ফারহান নামের একজন অভিজ্ঞ **System Analyst (সিস্টেম অ্যানালিস্ট)**-কে ডেকে আনলেন।

ফারহান কোম্পানিতে এসেই কিন্তু সরাসরি কম্পিউটারে বসে কোডিং শুরু করল না। সে তার ভেতরের গুণগুলো একে একে কাজে লাগাতে শুরু করল:

প্রথমত, ফারহান ছিল একজন দুর্দান্ত **Problem Solver (সমস্যা সমাধানকারী)**। সে কোম্পানির এই বিশাল বিশৃঙ্খলা দেখে ভয় না পেয়ে, এটিকে একটি চমৎকার চ্যালেঞ্জ হিসেবে গ্রহণ করল এবং বাস্তবসম্মত সমাধানের উপায় বা workable solutions বের করতে আনন্দ পেল।

তার ছিল চমৎকার **Communication Skills (যোগাযোগের দক্ষতা)**। ফারহান জানত যে সিস্টেম ঠিক করতে হলে সবার কথা শুনতে হবে। তাই সে যেমন একদিকে কোম্পানির মালিক ও টপ ম্যানেজমেন্টের সাথে ইংরেজিতে সুন্দর করে মিটিং করল, তেমনি অন্যদিকে একদম নিচুতলার ডেলিভারি ম্যান বা সাধারণ ইউজারদের সাথেও তাদের মতো করে মিশে গিয়ে অর্থবহভাবে (meaningfully) কথা বলল।

শুধু কথায় চিঁড়ে ভেজে না, তাই ফারহানের ছিল গভীর **Technical Knowledge (প্রযুক্তিগত জ্ঞান)**। সে লেটেস্ট প্রযুক্তি ও কম্পিউটারের ক্ষমতা সম্পর্কে বিস্তারিত জানত, যার ফলে ইউজারদের কী লাগবে সেটা বুঝে সে প্রোগ্রামারদের ঠিকমতো বুঝিয়ে দিতে পারল যে কী ধরনের কোড লিখতে হবে।

একই সাথে, ফারহান শুধুমাত্র একজন আইটি এক্সপার্ট ছিল না; তার ছিল দুর্দান্ত **Business Skills (ব্যবসায়িক দক্ষতা)**। সে খুব ভালো করেই বুঝত একটি ব্যবসার অপারেশন কীভাবে চলে এবং সে খুব সহজেই কোম্পানির ব্যবসায়িক চাহিদাকে (Business needs) আইটি বা টেকনিক্যাল রিকোয়ারমেন্টে রূপান্তর (Translate) করতে পারল।

কোম্পানিতে হাজার হাজার অগোছালো ডেটা আর রিপোর্ট ছড়িয়ে ছিল। ফারহান তখন তার **Critical Thinking Skills (বিশ্লেষণমূলক চিন্তার দক্ষতা)** ব্যবহার করল। সে সব ডেটা তুলনা করল, প্যাটার্ন (patterns) খুঁজলো এবং কার্যকারণ (cause-and-effect) ও লজিক ব্যবহার করে বের করল যে ভুলটা ডেলিভারি ম্যানদের নয়, বরং বারকোড প্রিন্ট হওয়ার সময় সিস্টেমে লজিক্যাল ভুল হচ্ছে।

এই পুরো প্রজেক্টের সময় ফারহান ছিল অত্যন্ত **Self-disciplined and Self-motivated (স্ব-শৃঙ্খল এবং স্ব-প্রণোদিত)**। তাকে কেউ বলে দিত না কী করতে হবে, সে নিজেই নিজের কাজের টাইমলাইন ঠিক করত এবং প্রজেক্টের অন্যান্য মানুষ ও রিসোর্সগুলোকে নিখুঁতভাবে পরিচালনা (manage and coordinate) করত।

সবশেষে, ক্লায়েন্টদের সাথে সম্পর্ক তৈরি ও বজায় রাখতে এবং কোম্পানির এত গুরুত্বপূর্ণ তথ্যের গোপনীয়তা রক্ষায় সে তার **Personal and Professional Ethics (ব্যক্তিগত ও পেশাগত নৈতিকতা)** শক্তভাবে মেনে চলেছিল।

ফারহানের এই সব অসাধারণ গুণের কারণেই 'কুইকড্রপ লজিস্টিকস' আজ আবার তাদের হারানো সুনাম ফিরে পেয়েছে!

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

একজন সফল সিস্টেম অ্যানালিস্টের প্রধান গুণাবলি বা **Qualities** নিচে বুলেট পয়েন্টে দেওয়া হলো:

- **Problem Solver (সমস্যা সমাধানকারী):** সিস্টেম অ্যানালিস্ট যেকোনো সমস্যাকে একটি চ্যালেঞ্জ হিসেবে দেখেন এবং সৃজনশীল ও কার্যকরী সমাধান (workable solutions) তৈরি করতে পছন্দ করেন।
- **Communication Skills (যোগাযোগের দক্ষতা):** প্রতিষ্ঠানের সব স্তরের মানুষের (অপারেশনাল স্টাফ থেকে শুরু করে সিনিয়র এক্সিকিউটিভ পর্যন্ত) সাথে সফলভাবে এবং দীর্ঘসময় ধরে যোগাযোগ রক্ষা করার অসাধারণ মৌখিক ও লিখিত দক্ষতা।
- **Technical Knowledge (প্রযুক্তিগত জ্ঞান):** অত্যাধুনিক প্রযুক্তি (state-of-the-art technologies), হার্ডওয়্যার ও সফটওয়্যার সম্পর্কে গভীর ধারণা থাকা, যাতে ইউজারদের চাহিদা বুঝে প্রোগ্রামারদের সঠিক গাইডলাইন দেওয়া যায়।
- **Business Skills (ব্যবসায়িক দক্ষতা):** ব্যবসার রীতিনীতি, অপারেশন এবং প্রসেসগুলো বুঝতে পারা এবং সেই ব্যবসায়িক চাহিদাকে (business needs) টেকনিক্যাল রিকোয়ারমেন্টে রূপান্তর করার ক্ষমতা।

- **Critical Thinking Skills (বিশ্লেষণমূলক চিন্তার দক্ষতা):** যেকোনো পরিস্থিতির ডেটা তুলনা করা (compare), প্যাটার্ন চেনা, কার্যকারণ (cause-and-effect) বিশ্লেষণ করা এবং সঠিক লজিক প্রয়োগ করে সিদ্ধান্ত নেওয়ার ক্ষমতা।
- **Self-disciplined & Self-motivated (স্ব-শৃঙ্খল এবং স্ব-প্রণোদিত):** নিজের কাজের প্রতি একাগ্র থাকা এবং প্রজেক্টের অন্যান্য মানুষ ও বিভিন্ন রিসোর্সকে সঠিকভাবে পরিচালনা (manage and coordinate) করার মানসিকতা।
- **Personal and Professional Ethics (ব্যক্তিগত ও পেশাগত নৈতিকতা):** ক্লায়েন্টদের সাথে বিশ্বস্ততার সম্পর্ক (client relationships) তৈরি করা এবং পেশাদারিত্ব ও সততা বজায় রেখে কাজ করা।

আশা করি ফারহানের এই গল্পটি মনে রাখলে একজন System Analyst-এর Qualities তোমাদের কাছে একেবারে পরিষ্কার থাকবে!

আজ আমি তোমাদের এই কোর্সের সবচেয়ে গুরুত্বপূর্ণ একটি বিষয় শেখাবো—**Systems Development Life Cycle (SDLC)**।

চলো, একটি পরিচিত ও মজাদার গল্পের মাধ্যমে এই পুরো প্রক্রিয়াটি বুঝে নিই।

গল্প: রফিকের 'ড্রিম বেকারি' এবং জাদুকরী ধাপগুলো

ধরে নাও, রফিকের 'ড্রিম বেকারি' নামে একটি দারুণ কেক ও পেস্ট্রির দোকান আছে। তার দোকানের কেকের এতই সুনাম যে প্রতিদিন উপচে পড়া ভিড় থাকে। কিন্তু ভিড় সামলাতে গিয়ে রফিক হিমশিম খাচ্ছে; ফোনে অর্ডার নিতে ভুল হচ্ছে, অনেক ক্রেতা ফিরে যাচ্ছে। রফিক বুঝতে পারল, তার একটি সুন্দর ই-কমার্স ওয়েবসাইট (e-commerce platform) দরকার, যেখানে ক্রেতারা অনলাইনে অর্ডার করতে পারবে।

এই কাজের জন্য রফিক ডাকল সাদিক নামের একজন অভিজ্ঞ সিস্টেম অ্যানালিস্টকে। সাদিক রফিককে বোঝাল, একটি ভালো সফটওয়্যার বা সিস্টেম হুট করে কোডিং করে বানানো যায় না। এটি ধাপে ধাপে একটি কাঠামোগত প্রক্রিয়ার (structured process) মাধ্যমে তৈরি করতে হয়, যাতে কোনো ভুল না হয় এবং সময় ও খরচ বাঁচে। এই ধাপে ধাপে সিস্টেম তৈরি করার প্রক্রিয়াটিকেই বলা হয় **Systems Development Life Cycle বা SDLC**।

সাদিক মূলত ৪টি ধাপে রফিকের প্রজেক্টটি সম্পন্ন করল:

ধাপ ১: Planning (পরিকল্পনা) সাদিক প্রথমেই কোড লেখা শুরু করল না। সে আগে রফিকের সাথে বসে প্রজেক্টের মূল লক্ষ্য বা গোল (project goals) ঠিক করল। সে বোঝার চেষ্টা করল কেন এই সিস্টেমটি বানানো দরকার। এরপর সে যাচাই করল এই ওয়েবসাইটটি বানানো আসলেই সম্ভব কি না—টেকনিক্যালি (তাদের কী প্রযুক্তি লাগবে), ইকোনমিক্যালি (রফিকের বাজেট আছে কি না), এবং অপারেশনালি (কর্মচারীরা এটি ব্যবহার করতে পারবে কি না)। এই সম্ভাবনা যাচাই বা **Feasibility Check** করার ধাপটিই হলো প্রজেক্টের একদম প্রাথমিক ধাপ, যাকে বলা হয় **Planning**।

ধাপ ২: Analysis (বিশ্লেষণ) পরিকল্পনা চূড়ান্ত হওয়ার পর সাদিক চলে গেল ক্রেতাদের এবং বেকারির কর্মচারীদের কাছে। সে তাদের প্রশ্ন করল—এই সিস্টেমে কে কে ব্যবহারকারী হবে (who), সিস্টেমটি কী কী কাজ করবে (what), এবং কখন ও কোথায় ব্যবহৃত হবে (where and when)। ক্রেতারা জানাল তাদের প্রোডাক্ট সার্চ করার অপশন, সিকিউর পেমেন্ট এবং অর্ডার ট্র্যাকিং সুবিধা লাগবে। এই যে সবার কাছ থেকে সিস্টেমের রিকোয়ারমেন্ট বা চাহিদাগুলো সংগ্রহ করা হলো, এই ধাপটির নাম **Analysis**।

ধাপ ৩: Design (নকশা) কী বানাতে হবে তা জানার পর, সাদিক এবার ঠিক করল সিস্টেমটি *কীভাবে* কাজ করবে (how system will work)। সে একটি ব্লুপ্রিন্ট বা আর্কিটেকচার তৈরি করল। ওয়েবসাইটের লেআউট কেমন হবে, ডাটাবেস (database) কোথায় থাকবে, ক্রেতা ও পণ্যের তথ্য কীভাবে সেভ হবে—এসব কিছুর একটি নিখুঁত নকশা তৈরি করা হলো। এই আর্কিটেকচার এবং ডাটাবেস নির্ধারণ করার ধাপটিই হলো **Design**।

ধাপ ৪: Implementation & Operation (বাস্তবায়ন ও পরিচালনা) নকশা হাতে পাওয়ার পর ডেভেলপাররা (Developers) এবার আসল কাজ শুরু করল। তারা শপিং কার্ট, ক্যাটালগ এবং পেমেন্ট মডিউলের কোডিং (Coding) করল। কোড লেখা শেষ হলে তা ভালোভাবে টেস্টিং (Testing) করে দেখা হলো কোনো বাগ (bug) বা এরর আছে কি না। সব ঠিকঠাক থাকলে ওয়েবসাইটটি সার্ভারে ইন্সটল করে লাইভ করে দেওয়া হলো (Installation)। আর সিস্টেম চালুর পর ক্রেতারা যখন এটি ব্যবহার করতে শুরু করল, তখন সাদিকের টিম নিয়মিত এর আপডেট ও ভুলত্রুটি ঠিক করার কাজ বা মেইনটেন্যান্স (Maintenance) চালিয়ে যেতে লাগল। এই শেষ ধাপটিকে বলা হয় **Implementation & Operation**।

সাদিকের এই সুশৃঙ্খল SDLC প্রক্রিয়ার কারণে রফিকের 'ড্রিম বেকারি' কোনো ঝামেলা ছাড়াই অনলাইনে দারুণ ব্যবসা শুরু করল!

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

- **SDLC (Systems Development Life Cycle) কী?** এটি হলো একটি স্টেপ-বাই-স্টেপ প্রসেস বা কাঠামোগত পদ্ধতি, যা একটি ইনফরমেশন সিস্টেমের প্ল্যান, ডিজাইন, বিল্ড, টেস্ট এবং মেইনটেইন করার জন্য ব্যবহৃত হয়। এটি ব্যবহার করলে কোনো রিকোয়ারমেন্ট বাদ পড়ে না, ঝুঁকি (risk) কমে এবং সময় ও খরচ নিয়ন্ত্রণে থাকে।
- **SDLC এর মূল ধাপসমূহ (Phases of SDLC):**
    - **Phase 1: Planning (প্ল্যানিং):** এটি প্রজেক্টের প্রথম ধাপ, যেখানে প্রজেক্টের লক্ষ্য বা গোল নির্ধারণ করা হয় এবং সিস্টেমটি তৈরি করা সম্ভব কি না তা যাচাই করার জন্য ফিজিবিলিটি স্টাডি (technical, economic, organizational feasibility) করা হয়।
    - **Phase 2: Analysis (অ্যানালাইসিস):** এই ধাপে সিস্টেমের রিকোয়ারমেন্ট বা ব্যবহারকারীদের চাহিদা সংগ্রহ করা হয়। এটি উত্তর দেয় যে সিস্টেমটি কে ব্যবহার করবে (who), সিস্টেমটি কী করবে (what), এবং কোথায় ব্যবহৃত হবে।

- **Phase 3: Design (ডিজাইন):** এই ধাপে সিস্টেমটি কীভাবে কাজ করবে তা নির্ধারণ করা হয়। এর মধ্যে সিস্টেম আর্কিটেকচার (হার্ডওয়্যার/সফটওয়্যার), ডাটাবেস ডিজাইন এবং ইউজার ইন্টারফেস লেআউটের নকশা অন্তর্ভুক্ত থাকে।
- **Phase 4: Implementation & Operation (ইমপ্লিমেন্টেশন এবং অপারেশন):** এটি হলো চূড়ান্ত ধাপ, যেখানে ডেভেলপাররা কোড লেখে, সিস্টেমটি টেস্ট করে, সার্ভারে ইন্সটল করে এবং তা ব্যবহারকারীদের জন্য উন্মুক্ত করে দেয়। পরবর্তীতে চলমান অপারেশন এবং সিস্টেমের আপডেট বা মেইনটেন্যান্স (maintenance) এই ধাপেরই অংশ।

আশা করি এই গল্পের মাধ্যমে SDLC বা Systems Development Life Cycle-এর কনসেপ্ট এবং এর প্রতিটি ধাপ তোমার কাছে একদম স্পষ্ট হয়ে গেছে!

সিস্টেম অ্যানালাইসিস অ্যান্ড ডিজাইনের (System Analysis and Design) এই চমৎকার যাত্রায় তোমাকে আবার স্বাগত জানাই। গত ২০ বছরের শিক্ষকতার অভিজ্ঞতায় আমি দেখেছি, সফটওয়্যার তৈরির বিভিন্ন মডেলগুলোর মধ্যে **Waterfall Model (ওয়াটারফল মডেল)** হলো সবচেয়ে প্রাচীন, ক্লাসিক এবং সহজে বোঝার মতো একটি পদ্ধতি।

চলো, এই মডেলটি আমরা একটি দারুণ এবং বাস্তবসম্মত গল্পের মাধ্যমে শিখে নিই।

গল্প: হাসান সাহেবের 'অ্যাকাউন্টিং সফটওয়্যার' এবং জলপ্রপাতের নিয়ম

ধরে নাও, হাসান সাহেবের একটি পুরোনো এবং প্রতিষ্ঠিত হার্ডওয়্যারের দোকান আছে। তার ব্যবসার হিসাব-নিকাশের নিয়মকানুন একেবারে নির্দিষ্ট; এখানে প্রতিদিনের বিক্রি, জমা এবং খরচের খাতা ছাড়া নতুন কোনো জটিলতা নেই। তিনি তার এই নির্দিষ্ট হিসাব রাখার জন্য একটি **Accounting Software** বানাতে চাইলেন।

এই কাজের জন্য তিনি সাদিক নামের একজন অত্যন্ত অভিজ্ঞ **System Analyst**-কে নিয়োগ দিলেন। হাসান সাহেব সাদিককে শুরুতেই তার ব্যবসার সমস্ত নিয়মকানুন এবং সফটওয়্যারের চাহিদাগুলো (Requirements) একেবারে পরিষ্কারভাবে লিখে দিলেন। তিনি সাদিককে বললেন, "আমার এই চাহিদাগুলোর বাইরে আর নতুন কোনো ফিচার লাগবে না।"

সাদিক তখন হাসান সাহেবকে বলল, "আপনার প্রজেক্টের যেহেতু কোনো পরিবর্তনশীল বা অনিশ্চিত চাহিদা নেই (no uncertain requirements) এবং এটি তুলনামূলক ছোট একটি প্রজেক্ট, তাই এই সফটওয়্যারটি বানাতে আমরা **Waterfall Model** ব্যবহার করব।"

হাসান সাহেব কৌতূহলী হয়ে জিজ্ঞেস করলেন, "ওয়াটারফল বা জলপ্রপাত মডেল কী?"

সাদিক হাসতে হাসতে খাতা-কলম বের করে বোঝাতে শুরু করল, "ধরুন, পাহাড়ের চূড়া থেকে একটি জলপ্রপাত বা ঝরনা নিচে পড়ছে। একবার পানি নিচে পড়ে গেলে তা কি আর উপরের দিকে ফিরে যেতে পারে? পারে না। আমাদের এই **Waterfall Model**-ও ঠিক তেমনি একটি **Linear-sequential life cycle model**। এখানে সফটওয়্যার তৈরির পুরো কাজটিকে কয়েকটি ধাপে (Phases) ভাগ করা হয় এবং একটি ধাপ পুরোপুরি শেষ হওয়ার পরই কেবল আমরা নিচের ধাপে যেতে পারি। কোনোভাবেই আগের ধাপে ফিরে যাওয়ার সুযোগ নেই।"

সাদিক এরপর কাজ শুরু করল। সে মূলত ৫টি ধাপে কাজ করল: ১. **Requirement Analysis (চাহিদা বিশ্লেষণ):** প্রথমেই সে হাসান সাহেবের সব চাহিদাগুলো সংগ্রহ করে চূড়ান্ত করল। ২. **System Design (নকশা):** এরপর সে সফটওয়্যারের আর্কিটেকচার এবং ডাটাবেস ডিজাইন করল। ৩. **Implementation বা Coding (বাস্তবায়ন):** ডিজাইন শেষ হলে

প্রোগ্রামাররা কোড লেখা শুরু করল। ৪. **Testing (টেস্টিং বা পরীক্ষণ):** পুরো কোডিং শেষ হওয়ার পর সফটওয়্যারটি টেস্ট করা হলো যে কোথাও কোনো বাগ (bug) বা এরর (error) আছে কি না। ৫. **Deployment and Maintenance (ডেলিভারি এবং রক্ষণাবেক্ষণ):** সবশেষে সফটওয়্যারটি হাসান সাহেবের কম্পিউটারে ইন্সটল করে দেওয়া হলো এবং এরপর শুধু নিয়মিত দেখাশোনা বা মেইনটেন্যান্সের কাজ চলল।

কিন্তু গল্পের একটি ছোট মোড় আছে! যখন ৩ নম্বর ধাপ অর্থাৎ **Coding** চলছিল, তখন হাসান সাহেব হঠাৎ সাদিককে ফোন করে বললেন, "সাদিক, আমার সফটওয়্যারে কি গ্রাহকদের সাথে চ্যাট (Chat) করার একটি অপশন যোগ করা যাবে?"

সাদিক বিনীতভাবে উত্তর দিল, "দুঃখিত হাসান সাহেব! আমরা **Waterfall Model** ব্যবহার করছি। আমাদের **Design** ধাপ আগেই শেষ হয়ে গেছে এবং আমরা এখন **Coding** করছি। এই মডেলে একবার নিচের ধাপে চলে এলে, পুনরায় আগের ধাপে গিয়ে নতুন কিছু যোগ করা প্রায় অসম্ভব এবং অত্যন্ত ব্যয়বহুল (Difficult to go back once a phase is completed)। তাছাড়া, একদম শেষ ধাপে না যাওয়া পর্যন্ত আপনি সফটওয়্যারটি চোখেও দেখতে পারবেন না (No working software is produced until late)।"

হাসান সাহেব বুঝতে পারলেন। যেহেতু তার মূল কাজ ছিল অ্যাকাউন্টিং এবং তার চাহিদাগুলো আগে থেকেই ফিক্সড ছিল, তাই সফটওয়্যারটি শেষ পর্যন্ত অত্যন্ত নিখুঁতভাবে এবং কোনো ঝামেলা ছাড়াই তৈরি হয়ে গেল।

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

গল্পের আলোকে **Waterfall Model**-এর মূল বিষয়গুলো নিচে বুলেট পয়েন্টে দেওয়া হলো:

- **Waterfall Model কী?** এটি হলো একটি **Linear-sequential flow** বা পর্যায়ক্রমিক মডেল। এখানে একটি ধাপ পুরোপুরি শেষ হলে তবেই পরবর্তী ধাপের কাজ শুরু হয়। একটি ধাপের আউটপুট পরবর্তী ধাপের ইনপুট হিসেবে কাজ করে।
- **Phases of Waterfall Model (ধাপসমূহ):**
    1. **Requirement Analysis:** সমস্ত চাহিদা সংগ্রহ ও বিশ্লেষণ করা।
    2. **System Design:** সিস্টেমের আর্কিটেকচার তৈরি করা।
    3. **Implementation (Coding):** কোড লেখা।
    4. **Testing:** কোডিং শেষ হওয়ার পর সফটওয়্যারটিতে কোনো ভুল আছে কি না তা পরীক্ষা করা।

5. **Deployment & Maintenance:** সিস্টেমটি গ্রাহককে বুঝিয়ে দেওয়া এবং নিয়মিত রক্ষণাবেক্ষণ করা।

- **কখন এই মডেল ব্যবহার করা উচিত?** যখন প্রজেক্টের সাইজ ছোট হয় এবং এর রিকোয়ারমেন্টগুলো (requirements) আগে থেকেই খুব ভালোভাবে জানা থাকে, নির্দিষ্ট (fixed) থাকে এবং ভবিষ্যতেও পরিবর্তনের কোনো সম্ভাবনা থাকে না (no uncertain requirements)।
- **Advantages (সুবিধাসমূহ):**
  - এটি অত্যন্ত সহজ, সাধারণ এবং ব্যবহার করা সোজা (Simple and easy to understand/use)।
  - এর প্রতিটি ধাপ এবং মাইলফলক খুব পরিষ্কারভাবে সংজ্ঞায়িত থাকে (Clearly defined stages)।
  - প্রতিটি ধাপে পরিষ্কার ডকুমেন্টেশন থাকে এবং এটি ম্যানেজ করা খুব সহজ।
- **Disadvantages (অসুবিধাসমূহ):**
  - **Inflexible (অনমনীয়):** পরে কোনো রিকোয়ারমেন্ট পরিবর্তন করতে চাইলে তা করা খুবই কঠিন (High risk if changes are needed later)।
  - **Late Testing:** কোডিং পুরোপুরি শেষ হওয়ার আগে টেস্টিং শুরু হয় না, ফলে বড় কোনো ভুল থাকলে তা একদম শেষে গিয়ে ধরা পড়ে (Late detection of major errors)।
  - **No Early Working Software:** প্রজেক্টের একদম শেষ পর্যায়ে যাওয়ার আগে গ্রাহক কোনো কাজ করা সফটওয়্যার দেখতে পারেন না।

আশা করি এই 'জলপ্রপাতের' গল্পের মাধ্যমে **Waterfall Model**-এর কনসেপ্ট, এর সুবিধা এবং অসুবিধাগুলো তোমার কাছে একেবারে জলের মতো পরিষ্কার হয়ে গেছে!

আজ আমি তোমাদের শেখাবো অত্যন্ত গুরুত্বপূর্ণ এবং একটু ভিন্নধর্মী একটি মডেল—**Spiral Model (স্পাইরাল মডেল)**।

চলো, এই মডেলটি আমরা একটি দারুণ গল্পের মাধ্যমে শিখে নিই।

গল্প: 'ট্রাস্ট ব্যাংক'-এর সিকিউর ব্যাংকিং অ্যাপ এবং জাদুকরী স্পাইরাল

ধরে নাও, 'ট্রাস্ট ব্যাংক' দেশের একটি স্বনামধন্য ব্যাংক। ব্যাংকের ম্যানেজিং ডিরেক্টর মিস্টার রহমান সিদ্ধান্ত নিলেন, তারা একটি অত্যাধুনিক মোবাইল ব্যাংকিং অ্যাপ তৈরি করবেন। কিন্তু এই অ্যাপটি সাধারণ কোনো অ্যাপ নয়; এতে ফিঙ্গারপ্রিন্ট স্ক্যানিং, রেটিনা স্ক্যান, ভয়েস রিকগনিশন এবং আর্টিফিশিয়াল ইন্টেলিজেন্স যুক্ত থাকবে।

মিস্টার রহমান খুব চিন্তিত। প্রজেক্টটি অত্যন্ত বড়, এর টেকনিক্যাল ঝুঁকি (Risk) অনেক বেশি, আর রিকোয়ারমেন্টগুলোও পুরোপুরি পরিষ্কার নয়। যদি কোনো হ্যাকার এই অ্যাপ হ্যাক করে, তবে ব্যাংকের কোটি কোটি টাকা এবং সুনাম নষ্ট হবে।

এই বিশাল এবং ঝুঁকিপূর্ণ কাজের জন্য মিস্টার রহমান ডাকলেন আমাদের পরিচিত ও অভিজ্ঞ সিস্টেম অ্যানালিস্ট সাদিককে। সাদিক সব শুনে বলল, "মিস্টার রহমান, আপনার এই প্রজেক্টটি অনেক বড় এবং ঝুঁকিপূর্ণ। এখানে আমরা Waterfall মডেল ব্যবহার করতে পারব না, কারণ সেখানে শেষে গিয়ে ভুল ধরা পড়লে বিশাল ক্ষতি হবে। আপনার এই প্রজেক্টের জন্য আমাদের এমন একটি মডেল লাগবে যা বারবার যাচাই করে এবং ঝুঁকি কমায়। আমরা ব্যবহার করব **Spiral Model (স্পাইরাল মডেল)**।"

মিস্টার রহমান অবাক হয়ে জিজ্ঞেস করলেন, "স্পাইরাল মডেল কী?"

সাদিক খাতা-কলম বের করে একটি শামুকের খোলসের মতো প্যাঁচানো বা স্পাইরাল চিত্র আঁকল। সে বোঝাতে শুরু করল, "নব্বইয়ের দশকে ব্যারি বোয়েম (Barry Boehm) এই মডেলটি প্রস্তাব করেন। এটি মূলত **Iterative development** (যেখানে কাজ বারবার চক্রাকারে হয়) এবং **Waterfall model**-এর (যেখানে কাজ ধাপে ধাপে হয়) একটি চমৎকার সংমিশ্রণ। এই মডেলের সবচেয়ে বড় বৈশিষ্ট্য হলো এটি **Risk Analysis** বা ঝুঁকি বিশ্লেষণের ওপর সবচেয়ে বেশি জোর দেয়। আমরা পুরো অ্যাপটি একসাথে না বানিয়ে, এই স্পাইরালের এক একটি চক্র বা লুপ (Loop) ধরে এগোব এবং প্রতি চক্রে একটি করে **Prototype** (নমুনা) তৈরি করব।"

সাদিক মূলত প্রতিটি চক্রে ৪টি ধাপে কাজ করল:

১. **Determine Objectives (উদ্দেশ্য নির্ধারণ ও প্ল্যানিং):** প্রথম ধাপে সাদিক ও মিস্টার রহমান ঠিক করলেন, এই চক্রে শুধু ইউজারদের লগ-ইন সিস্টেম তৈরি করা হবে। তারা রিকোয়ারমেন্ট বুঝলেন এবং প্ল্যান করলেন।

২. **Identify Risks (ঝুঁকি বিশ্লেষণ):** এটি স্পাইরাল মডেলের সবচেয়ে গুরুত্বপূর্ণ ধাপ। সাদিকের টিম গভীরভাবে বিশ্লেষণ করল—লগ-ইন সিস্টেমে হ্যাকাররা কীভাবে আক্রমণ করতে পারে? পাসওয়ার্ড চুরি হওয়ার ঝুঁকি কতটুকু? এই ঝুঁকিগুলো সমাধানের উপায় কী? তারা এই ধাপে সব ঝুঁকির সমাধান বের করল।

৩. **Engineering / Development and Testing (উন্নয়ন ও পরীক্ষণ):** এরপর ডেভেলপাররা লগ-ইন সিস্টেমের কোডিং করল এবং টেস্টিং করে একটি ছোট, ব্যবহারযোগ্য **Prototype 1** তৈরি করল।

৪. **Evaluation / Plan Next Iteration (মূল্যায়ন এবং পরবর্তী ধাপের পরিকল্পনা):** মিস্টার রহমান এই Prototype 1 নিজে ব্যবহার করে দেখলেন। তিনি কিছু নতুন ফিচারের কথা বললেন। তার মতামতের ভিত্তিতে সাদিকের টিম পরবর্তী স্পাইরাল চক্রের (যেমন: ফান্ড ট্রান্সফার ফিচার) জন্য প্ল্যানিং শুরু করল।

এভাবে প্রথম চক্রে লগ-ইন, দ্বিতীয় চক্রে ফান্ড ট্রান্সফার (Prototype 2), তৃতীয় চক্রে বিল পেমেন্ট (Prototype 3)—এভাবে স্পাইরালের চক্র ঘুরতে ঘুরতে এবং প্রতিটি ধাপে ঝুঁকি বিশ্লেষণ করতে করতে ট্রাস্ট ব্যাংকের পুরো অ্যাপটি তৈরি হয়ে গেল। যেহেতু প্রতি ধাপে ঝুঁকিগুলো আগেই চিহ্নিত করে সমাধান করা হয়েছিল, তাই অ্যাপটি হলো একদম ত্রুটিমুক্ত এবং অত্যন্ত সুরক্ষিত। মিস্টার রহমানও খুশি, আর সাদিকের প্রজেক্টও সফল!

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

গল্পের আলোকে **Spiral Model**-এর মূল বিষয়গুলো নিচে বুলেট পয়েন্টে দেওয়া হলো:

- **Spiral Model কী?**
  - এটি **Iterative development** (পুনরাবৃত্তিমূলক) এবং **Waterfall model**-এর (লিনিয়ার বা পর্যায়ক্রমিক) একটি সমন্বিত রূপ।
  - ১৯৯০-এর দশকে ব্যারি বোয়েম (Barry Boehm) এটি প্রথম প্রস্তাব করেন।
  - এই মডেলের সবচেয়ে বড় বৈশিষ্ট্য হলো এটি **Risk Analysis** (ঝুঁকি বিশ্লেষণ)-এর ওপর খুব বেশি জোর দেয়।
- **Phases of Spiral Model (প্রতিটি চক্র বা Iteration-এর ৪টি ধাপ):**

1. **Determine Objectives/Planning:** প্রজেক্টের উদ্দেশ্য, রিকোয়ারমেন্ট এবং বিকল্পগুলো চিহ্নিত করা।
2. **Identify Risks (Risk Analysis):** প্রজেক্টের ঝুঁকিগুলো (যেমন—টেকনিক্যাল বা খরচের ঝুঁকি) চিহ্নিত করা এবং তা কমানোর উপায় বের করা।
3. **Engineering (Development and Testing):** কোড লেখা, টেস্টিং করা এবং একটি কার্যকরী **Prototype** বা প্রাথমিক রূপ তৈরি করা।
4. **Evaluation (Review/Plan next iteration):** গ্রাহক বা ক্লায়েন্ট সেই প্রোটোটাইপটি মূল্যায়ন করেন এবং এর ভিত্তিতে পরবর্তী চক্রের পরিকল্পনা করা হয়।

- **Advantages (সুবিধাসমূহ):**
  - **Risk Management (ঝুঁকি ব্যবস্থাপনা):** প্রজেক্টের ঝুঁকিপূর্ণ অংশগুলো আগেই তৈরি ও যাচাই করা যায় বলে বড় ধরনের ফেইলিওরের ঝুঁকি কমে যায়।
  - **Flexibility:** রিকোয়ারমেন্ট পরিবর্তন হলে তা সহজেই এই মডেলে মানিয়ে নেওয়া যায় (Changing requirements can be accommodated)।
  - **Early Working Prototype:** ইউজাররা প্রজেক্টের প্রথম দিক থেকেই প্রোটোটাইপ (Prototype) বা সফটওয়্যারটির কাজ দেখতে পান।
  - বড় এবং জটিল প্রজেক্টের জন্য এটি সবচেয়ে উপযুক্ত।
- **Disadvantages (অসুবিধাসমূহ):**
  - **Complex Management (জটিল ব্যবস্থাপনা):** এই মডেলের প্রসেস বেশ জটিল, তাই ম্যানেজ করা কঠিন।
  - **High Cost (অধিক খরচ):** এটি ছোট বা কম ঝুঁকির প্রজেক্টের জন্য উপযুক্ত নয়, কারণ এটি অত্যন্ত ব্যয়বহুল হতে পারে।
  - **Excessive Documentation:** অনেকগুলো ধাপ এবং প্রোটোটাইপ থাকে বলে এতে প্রচুর কাগজপত্রের বা ডকুমেন্টের প্রয়োজন হয়।
  - প্রজেক্ট ঠিক কবে শেষ হবে, তা অনেক সময় আগে থেকে সঠিকভাবে বলা যায় না (Spiral may go on indefinitely)।

আশা করি এই 'ট্রাস্ট ব্যাংকের' গল্পের মাধ্যমে **Spiral Model** কীভাবে কাজ করে এবং কেন এটি বড় প্রজেক্টের জন্য এত জনপ্রিয়, তা তোমার কাছে পরিষ্কার হয়েছে!

আজ আমরা শিখব অত্যন্ত শৃঙ্খলাবদ্ধ এবং টেস্টিং বা পরীক্ষণের ওপর সবচেয়ে বেশি জোর দেওয়া একটি মডেল—**V-Model (ভি-মডেল)**।

চলো, একটি জীবনঘনিষ্ঠ গল্পের মাধ্যমে বিষয়টি বুঝে নিই।

গল্প: 'লাইফকেয়ার হাসপাতাল'-এর পেশেন্ট মনিটরিং সিস্টেম এবং তারিকের 'ভি (V)'

ধরে নাও, শহরের সবচেয়ে বড় হাসপাতাল 'লাইফকেয়ার হাসপাতাল'-এর ডিরেক্টর ডা. শফিক একটি নতুন 'অটোমেটেড পেশেন্ট মনিটরিং সিস্টেম' বানাতে চান। এই সিস্টেমটি আইসিইউ (ICU)-তে থাকা রোগীদের হার্টবিট, পালস এবং অক্সিজেনের মাত্রা মনিটর করবে এবং কোনো সমস্যা হলে সাথে সাথে ডাক্তারদের অ্যালার্ম দিয়ে জানাবে।

ডা. শফিক এই গুরুত্বপূর্ণ প্রজেক্টের দায়িত্ব দিলেন আমাদের সেই পরিচিত ও অভিজ্ঞ **System Analyst** তারিককে। ডা. শফিক অত্যন্ত চিন্তিত মুখে তারিককে বললেন, "তারিক, এই সফটওয়্যারে কোনোভাবেই কোনো ভুল বা এরর (Error) থাকা চলবে না। সফটওয়্যারের ছোট্ট একটি ভুলের কারণে কোনো রোগীর জীবন বিপন্ন হতে পারে! আমরা একদম শেষে গিয়ে যদি টেস্টিং করি আর বড় ভুল ধরা পড়ে, তাহলে তো সর্বনাশ হয়ে যাবে।"

তারিক সব শুনে বলল, "ডাক্তার সাহেব, একদম চিন্তা করবেন না। আপনার প্রজেক্টের রিকোয়ারমেন্টগুলো একদম পরিষ্কার, কিন্তু এখানে নিখুঁত হওয়াটা সবচেয়ে বেশি জরুরি। তাই আমরা সাধারণ Waterfall মডেল ব্যবহার করব না, কারণ সেখানে টেস্টিং হয় একদম শেষে। আমরা ব্যবহার করব **V-Model**, যার মানে হলো **Verification and Validation model**।"

ডা. শফিক কৌতূহলী হয়ে বললেন, "ভি-মডেল (V-Model) আবার কী?"

তারিক তখন হোয়াইটবোর্ডে ইংরেজি 'V' (ভি) অক্ষরটি এঁকে বোঝাতে শুরু করল: "ডাক্তার সাহেব, এই মডেলটি আসলে Waterfall মডেলেরই একটি উন্নত রূপভেদ (variation), যেখানে **Testing (টেস্টিং)**-এর ওপর সবচেয়ে বেশি মনোযোগ দেওয়া হয়। আমরা ইংরেজি 'V' অক্ষরের বাম দিকের ঢাল (left-hand slope) বেয়ে নিচে নামব এবং ডান দিকের ঢাল (rightward slope) বেয়ে উপরে উঠব।

বামের ঢাল (Left-hand slope): আমরা যখন V-এর বাম দিক দিয়ে নিচে নামব, তখন আমরা শুধু সিস্টেম ডিজাইনই করব না, সাথে সাথে সেই ডিজাইনের টেস্টিং কীভাবে হবে তার প্ল্যানও তৈরি করে ফেলব। যেমন:

- যখন আমরা আপনার কাছ থেকে **Requirements Analysis** করব, ঠিক তখনই আমরা একদম শেষের **Acceptance test design** তৈরি করে রাখব।
- এরপর যখন আমরা **System Design** করব, তখনই আমরা **System test design** তৈরি করে ফেলব।
- এভাবে ছোট ছোট মডিউল বা **Unit** ডিজাইন করার সময়ই আমরা **Unit test design** তৈরি করে রাখব।

নিচের অংশ (Base of the V): V-এর একদম নিচে বা তলানিতে এসে প্রোগ্রামাররা **Coding (কোডিং)** বা ইমপ্লিমেন্টেশনের কাজ করবে।

ডানের ঢাল (Right-hand slope): কোডিং শেষ হওয়ার পর আমরা V-এর ডান দিক দিয়ে উপরে উঠতে শুরু করব। বাম দিকে আমরা যেই টেস্ট প্ল্যানগুলো বানিয়েছিলাম, এবার সেগুলো ধাপে ধাপে প্রয়োগ করব। প্রথমে সবচেয়ে ছোট অংশের **Unit testing**, এরপর সেগুলো জোড়া লাগিয়ে **Integration testing**, তারপর পুরো সিস্টেমের **System testing** এবং একদম শেষে আপনার উপস্থিতিতে **Acceptance testing** সম্পন্ন করব।

প্রতিটি ডিজাইনের ধাপের সাথে তার নিজস্ব টেস্টিংয়ের ধাপটি সরাসরি যুক্ত থাকে (clearly linked)। এতে করে কোনো ভুল হওয়ার সম্ভাবনা প্রায় শূন্যের কোঠায় নেমে আসে।"

ডা. শফিক এই চমৎকার ও নিখুঁত প্ল্যান দেখে অত্যন্ত মুগ্ধ হলেন। যদিও এই মডেলটি Waterfall-এর মতোই কিছুটা অনমনীয় বা রিজিড (rigid) ছিল, কিন্তু যেহেতু হাসপাতালের রিকোয়ারমেন্ট আগে থেকেই ফিক্সড ছিল, তাই V-Model ব্যবহার করে একদম ত্রুটিমুক্ত এবং হাই-কোয়ালিটি একটি পেশেন্ট মনিটরিং সিস্টেম তৈরি হলো!

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

গল্পের আলোকে **V-Model (ভি-মডেল)**-এর মূল বিষয়গুলো নিচে বুলেট পয়েন্টে দেওয়া হলো:

- **V-Model কী?**
  - V-Model-এর পূর্ণরূপ হলো **Verification and Validation model**।
  - এটি মূলত Waterfall ডেভেলপমেন্টেরই একটি ভ্যারিয়েশন বা রূপভেদ, তবে এখানে **Testing (টেস্টিং বা পরীক্ষণ)**-এর ওপর সবচেয়ে বেশি জোর বা মনোযোগ দেওয়া হয়।
- **V-Model কীভাবে কাজ করে?**

- **Left-hand slope (বামের ঢাল):** এই ধাপে সিস্টেমের রিকোয়ারমেন্ট স্পেসিফিকেশন এবং কম্পোনেন্ট ডিজাইন করা হয়। এর সবচেয়ে বড় বৈশিষ্ট্য হলো—প্রতিটি ডিজাইন ধাপের সাথে সাথেই তার টেস্টিং প্ল্যান বা ডিজাইনও তৈরি করা হয়।
- **Base of the V (ভি-এর তলানি):** এখানে সিস্টেমের মূল **Coding (কোডিং)** বা বাস্তবায়নের কাজ করা হয়।
- **Right-hand slope (ডানের ঢাল):** কোডিং শেষে এই ধাপে ক্রমান্বয়ে টেস্টিংগুলো করা হয়—**Unit testing, Integration testing, System testing, এবং Acceptance testing।**

- **প্রধান বৈশিষ্ট্য (Key Concept):**
  - টেস্টিংয়ের প্রতিটি লেভেল অ্যানালাইসিস বা ডিজাইনের একটি নির্দিষ্ট অংশের সাথে সরাসরি যুক্ত (clearly linked) থাকে। এটি সফটওয়্যারের উচ্চমান বা High quality এবং টেস্টিংয়ের সর্বোচ্চ কার্যকারিতা নিশ্চিত করে।
- **অসুবিধা (Disadvantage):**
  - যেহেতু এটি Waterfall মডেলের একটি রূপভেদ, তাই এটি সেই মডেলের মতোই অনমনীয় বা **Rigid।** অর্থাৎ একবার একটি ধাপ পেরিয়ে গেলে বা কোডিং শুরু হয়ে গেলে, রিকোয়ারমেন্ট পরিবর্তন করা বেশ কঠিন হয়ে দাঁড়ায়।

আশা করি 'লাইফকেয়ার হাসপাতাল'-এর এই গল্পের মাধ্যমে V-Model-এর ধারণা এবং কেন এটি অত্যন্ত সেনসিটিভ বা ঝুঁকিপূর্ণ প্রজেক্টে এত দরকারি, তা তোমার কাছে একেবারে পরিষ্কার হয়ে গেছে!

আজ আমি তোমাদের শেখাবো বর্তমান সময়ের সবচেয়ে জনপ্রিয় একটি পদ্ধতি—**Agile Model (অ্যাজাইল মডেল)**।

চলো, এই পদ্ধতিটি একটি দারুণ এবং বাস্তবসম্মত গল্পের মাধ্যমে শিখে নিই।

গল্প: জায়েদের 'ফুডি-বাডি' অ্যাপ এবং সাদিকের 'অ্যাজাইল (Agile)' ম্যাজিক

ধরে নাও, জায়েদ একজন তরুণ উদ্যোক্তা, যে 'ফুডি-বাডি' নামে একটি নতুন ফুড ডেলিভারি ও সোশ্যাল নেটওয়ার্কিং প্ল্যাটফর্ম বানাতে চায়। কিন্তু জায়েদের সমস্যা হলো, ক্রেতারা ঠিক কী ধরনের ফিচার পছন্দ করবে তা সে পুরোপুরি জানে না এবং এই ধরনের ব্যবসার রিকোয়ারমেন্ট বা চাহিদাও প্রতিনিয়ত বদলাতে থাকে। তার মানে, প্রজেক্টের রিকোয়ারমেন্টগুলো এখানে অনিশ্চিত (uncertain) এবং প্রযুক্তিগত ঝুঁকিও (technical risk) অনেক বেশি।

জায়েদ এই কাজের জন্য আমাদের পরিচিত সিস্টেম অ্যানালিস্ট সাদিকের কাছে গেল। সাদিক জানাল, "যেহেতু আপনার প্রজেক্টের চাহিদাগুলো ঘন ঘন বদলাতে পারে এবং ইউজারদের ফিডব্যাক অনুযায়ী নতুন ফিচার যোগ করতে হবে, তাই আমরা এখানে সাধারণ Waterfall মডেল ব্যবহার করব না; আমরা **Agile Model** ব্যবহার করব।"

জায়েদ কৌতূহলী হয়ে জানতে চাইল, "অ্যাজাইল মডেল (Agile Model) কী?"

সাদিক বুঝিয়ে বলল, "অ্যাজাইল মেথড হলো এমন একটি সফটওয়্যার ডেভেলপমেন্ট পদ্ধতি যা মানুষ ও যোগাযোগের ওপর নির্ভরশীল এবং অত্যন্ত ফ্লেক্সিবল (Flexible) বা নমনীয়। এটি নির্দিষ্ট কিছু ভ্যালু (values), প্রিন্সিপাল (principles), এবং কোর প্র্যাকটিস (core practices)-এর ওপর ভিত্তি করে তৈরি। আমরা পুরো অ্যাপটি একসাথে না বানিয়ে, ছোট ছোট অংশে (Incrementally) ভাগ করে বারবার চক্রাকারে (Iterative cycles) তৈরি করব।"

সাদিক ও তার টিম এই প্রজেক্টে অ্যাজাইলের ৪টি মূল মূল্যবোধ বা **Values** মেনে কাজ শুরু করল: ১. **Communication (যোগাযোগ):** টিমের সবার মধ্যে এবং ক্লায়েন্টের সাথে নিয়মিত যোগাযোগ রাখা। ২. **Simplicity (সহজবোধ্যতা):** শুরুতে সবচেয়ে সহজ ও সাধারণ সমাধান দিয়ে কাজ শুরু করা। ৩. **Feedback (মতামত):** খুব দ্রুত সফটওয়্যারের কাজের ফলাফল বা ফিডব্যাক গ্রহণ করা। ৪. **Courage (সাহস):** কোনো কোড ভুল হলে তা ফেলে দিয়ে নতুন করে শুরু করার সাহস রাখা।

এই ভ্যালুগুলো মেনে সাদিকের টিম অ্যাজাইলের ৪টি জাদুকরী **Core Practices (মূল অনুশীলন)** প্রয়োগ করল:

- প্রথমত, তারা পুরো অ্যাপটি এক বছরে রিলিজ করার বদলে মাত্র কয়েক সপ্তাহের মধ্যে একটি বেসিক ভার্সন তৈরি করে ফেলল, যাকে বলা হয় **Short Releases**।
- তারা জায়েদের রেস্টুরেন্টের একজন অভিজ্ঞ ম্যানেজারকে সবসময় টিমের সাথে রাখল, যাকে বলা হয় **Onsite Customer**, যাতে যেকোনো সময় তার কাছ থেকে ব্যবসার রিকোয়ারমেন্ট বা 'User Stories' শোনা যায়।
- টিমের প্রোগ্রামাররা একা কাজ না করে দু'জন মিলে একসাথে কোড করা শুরু করল, যা হলো **Pair Programming**; এতে ভুলের মাত্রা অনেক কমে গেল।
- আর তারা সপ্তাহে ৪০ ঘন্টার বেশি কাজ করত না (**40-hour Workweek**), যাতে কেউ ক্লান্ত হয়ে ভুল না করে বা কাজের মান খারাপ না হয়।

কাজ করতে গিয়ে হঠাৎ জায়েদ বলল, "সাদিক, আগামী সপ্তাহের মধ্যেই একটি নতুন 'চ্যাট ফিচার' যুক্ত করতে হবে!" সাদিক তখন প্রজেক্টের ৪টি কন্ট্রোল ভেরিয়েবল—**Time (সময়), Cost (খরচ), Quality (মান), এবং Scope (পরিধি)**-এর হিসাব করল। সাদিক জায়েদকে বলল, "সময় যেহেতু কম, আমরা কোয়ালিটি বা মান খারাপ করতে পারব না; তার চেয়ে বরং আমরা অন্য একটি কম গুরুত্বপূর্ণ ফিচার আপাতত বাদ দিই অর্থাৎ Scope কমিয়ে ফেলি।"

এভাবে এক্সপ্লোরেশন, প্ল্যানিং থেকে শুরু করে বারবার ইটারেশন (Iterations) ও ফিডব্যাক গ্রহণের মাধ্যমে অত্যন্ত দ্রুত ও সফলভাবে 'ফুডি-বাডি' অ্যাপটি তৈরি হয়ে গেল। জায়েদও খুব খুশি, কারণ সে যেমনটা চেয়েছিল, ঠিক তেমনই একটি ডাইনামিক অ্যাপ সে পেয়েছে!

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

গল্পের আলোকে **Agile Model**-এর মূল বিষয়গুলো নিচে বুলেট পয়েন্টে দেওয়া হলো:

- **Agile Model কী?** এটি একটি নমনীয় (Adaptive), দ্রুতগামী (Speedy) এবং পুনরাবৃত্তিমূলক (Iterative) সফটওয়্যার ডেভেলপমেন্ট মেথডলজি, যা প্রজেক্টের পরিবর্তনের সাথে খুব সহজেই খাপ খাইয়ে নিতে পারে।
- **৪টি মূল মূল্যবোধ (Four Values of Agile):** সফল অ্যাজাইল ডেভেলপমেন্টের জন্য ৪টি ভ্যালু অপরিহার্য—**Communication** (যোগাযোগ), **Simplicity** (সহজবোধ্যতা), **Feedback** (মতামত), এবং **Courage** (সাহস)।
- **৪টি মূল অনুশীলন (Four Core Practices):** অ্যাজাইল মডেলকে অন্যান্য মডেল থেকে যা আলাদা করে, তা হলো এর ৪টি প্র্যাকটিস: ১. **Short Releases:** দ্রুত সফটওয়্যারের ছোট ছোট আপডেট রিলিজ করা। ২. **40-hour workweek:** ডেভেলপারদের ওভারওয়ার্ক বা বার্নআউট থেকে বাঁচাতে সপ্তাহে ৪০ ঘন্টার বেশি কাজ না করা। ৩.

Onsite customer: ক্লায়েন্ট বা কাস্টমারকে সরাসরি ডেভেলপমেন্ট টিমের সাথে যুক্ত রাখা। ৪. **Pair programming:** দুজন প্রোগ্রামার একসাথে মিলে কোড ও টেস্টিং করা।

- **৪টি রিসোর্স ভেরিয়েবল (Resource Control Variables):** অ্যাজাইল মডেলে প্রজেক্ট ম্যানেজমেন্টের জন্য ৪টি ভেরিয়েবল ব্যালেন্স করতে হয়—**Time, Cost, Quality, এবং Scope**।
- **অ্যাজাইল প্রক্রিয়ার ধাপ (Agile Development Process Stages):** অ্যাজাইল প্রসেসের মূলত ৫টি ধাপ থাকে: Exploration, Planning, Iterations to the first release, Productionizing, এবং Maintenance।
- **সুবিধাসমূহ (Advantages):**
    - রিকোয়ারমেন্ট পরিবর্তন হলেও সহজেই মানিয়ে নেওয়া যায় (Handles changing requirements easily)।
    - খুব দ্রুত এবং নিয়মিত ইউজারদের ফিডব্যাক নিয়ে সফটওয়্যার তৈরি করা যায় (Fast delivery and user-focused)।
    - ডেভেলপার এবং ক্লায়েন্টের মধ্যে শক্তিশালী কোলাবোরেশন বা যোগাযোগ তৈরি হয়।
- **অসুবিধাসমূহ (Disadvantages):**
    - এই মডেলে ডকুমেন্টের (Documentation) ওপর কম জোর দেওয়া হয়।
    - কাজটি সফল করতে হলে উচ্চ ক্ষমতাসম্পন্ন ও দক্ষ টিমের প্রয়োজন হয় (Requires highly skilled teams)।
- **কখন ব্যবহার করবেন? (Scenario-based Usage):** যখন প্রজেক্টের রিকোয়ারমেন্টগুলো অনিশ্চিত থাকে, মার্কেট ট্রেন্ড দ্রুত বদলায় এবং কাস্টমারদের ফিডব্যাকের ওপর ভিত্তি করে বারবার নতুন ফিচার রিলিজ করতে হয় (যেমন—Social Networking Platform বা Online Learning Platform), তখন Agile Model সবচেয়ে ভালো কাজ করে।

আশা করি 'ফুডি-বাডি' অ্যাপের এই গল্পের মাধ্যমে **Agile Model**-এর ধারণা তোমার কাছে একেবারেই জলের মতো পরিষ্কার হয়ে গেছে!

গত ২০ বছর ধরে পড়াতে গিয়ে আমি দেখেছি, ছাত্র-ছাত্রীরা সবচেয়ে বেশি যে দুটি বিষয় নিয়ে গোলমাল পাকায় তা হলো **JAD (Joint Application Design)** এবং **RAD (Rapid Application Development)**। নাম দুটি শুনতে কাছাকাছি মনে হলেও, এদের কাজ কিন্তু সম্পূর্ণ আলাদা!

চলো, এই কনফিউশন দূর করার জন্য আজ আমরা একটি নতুন এবং খুব বাস্তবসম্মত গল্প শুনি।

গল্প: সাদিকের দুই ক্লায়েন্ট–রহমান সাহেব এবং তরুণ উদ্যোক্তা কামাল

আমাদের পরিচিত অভিজ্ঞ সিস্টেম অ্যানালিস্ট সাদিকের কাছে একদিন দুজন ভিন্ন ধরনের ক্লায়েন্ট এলেন।

প্রথমজন হলেন রহমান সাহেব। তিনি শহরের সবচেয়ে বড় 'কেয়ার হাসপাতাল'-এর মালিক। রহমান সাহেব তার হাসপাতালের জন্য একটি নতুন ম্যানেজমেন্ট সিস্টেম বানাতে চান। কিন্তু সমস্যা হলো, হাসপাতালের ডাক্তাররা চাইছেন এক ধরনের ফিচার, নার্সরা চাইছেন অন্যরকম, আর ম্যানেজমেন্টের চিন্তা শুধু খরচ আর সিকিউরিটি নিয়ে। সাদিক বুঝতে পারল, একা একা সবার ইন্টারভিউ (one-on-one interviews) নিতে গেলে মাসের পর মাস পার হয়ে যাবে এবং ভুল বোঝাবুঝি বা misinterpretation হওয়ার বিশাল সম্ভাবনা আছে।

এই সমস্যা সমাধানের জন্য সাদিক **JAD (Joint Application Design)** পদ্ধতির আশ্রয় নিল। সে রহমান সাহেবকে বলল, "আমরা একটি ৩ দিনের টানা ওয়ার্কশপের আয়োজন করব।" সাদিক একটি কনফারেন্স রুমে সবাইকে ডাকল। সেখানে উপস্থিত ছিলেন:

- **Managers (ম্যানেজার):** রহমান সাহেব নিজে, যিনি সিদ্ধান্ত নেবেন।
- **Users (ব্যবহারকারী):** হেড ডাক্তার এবং নার্সরা, যারা সিস্টেমটি ব্যবহার করবেন।
- **Analysts (অ্যানালিস্ট):** সাদিক এবং তার টেকনিক্যাল টিম।
- **Facilitator (ফ্যাসিলিটেটর):** একজন নিরপেক্ষ ব্যক্তি, যিনি এই ৩ দিনের মিটিংটি পরিচালনা করলেন এবং সবার মতামতের সমন্বয় করলেন।
- **Scribe (স্ক্রাইব):** যিনি সবার আলোচনার নোটগুলো লিখে রাখলেন।

এই ৩ দিনের নিবিড় আলোচনার মাধ্যমে হাসপাতালের সবার সব দ্বন্দ্ব দূর হয়ে গেল এবং সবাই মিলে সিদ্ধান্ত নিল সিস্টেমটি কেমন হবে। অর্থাৎ, **JAD** কোনো কোডিং করার পদ্ধতি নয়, এটি হলো ব্যবহারকারী, ম্যানেজার এবং অ্যানালিস্টদের একসাথে বসিয়ে অত্যন্ত দ্রুত ও নির্ভুলভাবে সিস্টেমের রিকোয়ারমেন্টস (Requirements) বা চাহিদাগুলো বের করার একটি দুর্দান্ত কৌশল।

অন্যদিকে, সাদিকের দ্বিতীয় ক্লায়েন্ট হলেন তরুণ উদ্যোক্তা কামাল। কামাল 'কুইকবাইট' নামে একটি নতুন ফাস্ট-ফুড ডেলিভারি ব্যবসা শুরু করেছে। তার কোনো বড় টিম নেই, কিন্তু আগামী এক মাসের মধ্যে তার একটি মোবাইল অ্যাপ দরকার, নইলে প্রতিযোগী কোম্পানি বাজার দখল করে নেবে।

এখানে সাদিক JAD ব্যবহার করল না, কারণ কামালের কোনো বড় টিম নেই যার সাথে সে ৩ দিন মিটিং করবে। এখানে দরকার প্রচণ্ড দ্রুতগতিতে সফটওয়্যার বানানো। তাই সাদিক বেছে নিল **RAD (Rapid Application Development)**।

সাদিক কামালের কাছ থেকে প্রাথমিক রিকোয়ারমেন্টস শুনেই দ্রুত একটি ডেমো বা **Prototype (প্রোটোটাইপ)** বানিয়ে ফেলল। কামাল সেই প্রোটোটাইপটি চালিয়ে দেখল এবং কিছু ফিডব্যাক দিল। সাদিকের টিম কোনো দীর্ঘমেয়াদী রিকোয়ারমেন্ট অ্যানালাইসিসে না গিয়ে, CASE Tools ব্যবহার করে খুব দ্রুত সেই ফিডব্যাক অনুযায়ী অ্যাপটি আপডেট করে দিল। মাত্র কয়েকটি ইটারেশন (iterations) বা চক্রের পরেই কামালের অ্যাপটি মার্কেটে রিলিজ করার জন্য তৈরি হয়ে গেল!

অর্থাৎ, **RAD** হলো একটি সম্পূর্ণ সফটওয়্যার ডেভেলপমেন্ট মেথডলজি, যার মূল লক্ষ্য হলো প্রোটোটাইপিং এবং ইউজার ফিডব্যাকের মাধ্যমে খুব দ্রুত একটি কাজ করা সফটওয়্যার (functioning information system) ডেলিভারি দেওয়া।

গল্পের মূল কথা হলো: রহমান সাহেবের বড় টিমের কাছ থেকে নিখুঁত রিকোয়ারমেন্ট জানার জন্য সাদিক ব্যবহার করল **JAD**। আর কামালের জন্য দ্রুত কোডিং করে সফটওয়্যার বানানোর জন্য ব্যবহার করল **RAD**।

Key Information Summarized (মূল তথ্যের সারসংক্ষেপ):

Joint Application Design (JAD):

- **সংজ্ঞা (Definition):** এটি একটি স্ট্রাকচারড প্রসেস বা ওয়ার্কশপ, যেখানে ইউজার, ম্যানেজার এবং সিস্টেম অ্যানালিস্টরা একসাথে বসে কয়েকদিনের নিবিড় মিটিংয়ের মাধ্যমে সিস্টেমের রিকোয়ারমেন্টস (Requirements) বা চাহিদাগুলো নির্ধারণ করেন। এটি প্রথম IBM তৈরি করেছিল।
- **অংশগ্রহণকারী (Key Participants):** সেশন লিডার বা Facilitator, Users, Managers, Analysts, এবং Scribe (যিনি নোট নেন)।
- **সুবিধাসমূহ (Advantages):**

    - ইউজারদের সরাসরি অংশগ্রহণ থাকে বলে সিস্টেম ইউজারদের চাহিদা খুব ভালোভাবে পূরণ করে।
    - ঐতিহ্যবাহী ইন্টারভিউয়ের তুলনায় এটি দ্রুত রিকোয়ারমেন্ট সংগ্রহ করে (প্রায় ১৫% সময় বাঁচায়)।
    - ভুল বোঝাবুঝি (miscommunication) কম হয় এবং সবার সামনে আলোচনা হয় বলে দ্রুত দ্বন্দ্ব (conflicts) নিরসন হয়।
- **অসুবিধাসমূহ (Disadvantages):**
    - এটি সময়সাপেক্ষ; কারণ প্রতিষ্ঠানের সব গুরুত্বপূর্ণ মানুষকে একসাথে ২-৪ দিনের জন্য এক রুমে পাওয়া খুব কঠিন।
    - খুবই দক্ষ একজন Facilitator-এর প্রয়োজন হয়।

Rapid Application Development (RAD):

- **সংজ্ঞা (Definition):** এটি একটি সফটওয়্যার ডেভেলপমেন্ট অ্যাপ্রোচ, যেখানে প্রোটোটাইপিং (Prototyping), ইউজার ফিডব্যাক এবং CASE টুলস ব্যবহার করে অত্যন্ত দ্রুতগতিতে একটি সম্পূর্ণ সফটওয়্যার তৈরি করা হয়।
- **ধাপসমূহ (Phases):** এর ৪টি মূল ধাপ রয়েছে—Requirements Planning, RAD Design Workshop, Construction (বা Build the System), এবং Implementation।
- **সুবিধাসমূহ (Advantages):**
    - সফটওয়্যার ডেভেলপমেন্টের সময় ও খরচ ব্যাপকভাবে কমে যায়।
    - খুব দ্রুত প্রোটোটাইপ তৈরি হয় বলে ইউজাররা শুরুতেই ফিডব্যাক দিতে পারে।
    - সিস্টেমের কম্পোনেন্টগুলো পুনরায় ব্যবহার (reusability) করা যায়।
- **অসুবিধাসমূহ (Disadvantages):**
    - বড় বা খুব জটিল প্রজেক্টের জন্য এটি উপযুক্ত নয়।
    - অত্যন্ত দক্ষ ডিজাইনার এবং ডেভেলপারের প্রয়োজন হয়।
    - এটি লং-টার্ম স্ট্র্যাটেজির চেয়ে তাৎক্ষণিক সমাধানের ওপর বেশি জোর দেয়।

JAD vs RAD (মূল পার্থক্য):

- **উদ্দেশ্য:** JAD মূলত রিকোয়ারমেন্টস বা চাহিদা সংগ্রহ করার একটি টেকনিক বা ওয়ার্কশপ। অন্যদিকে, RAD হলো একটি সম্পূর্ণ সফটওয়্যার তৈরির মেথডলজি (যেখানে রিকোয়ারমেন্ট থেকে শুরু করে ইমপ্লিমেন্টেশন পর্যন্ত সবকিছু থাকে)।
- **ফলাফল:** JAD-এর চূড়ান্ত ফলাফল হলো নিখুঁত 'Requirements Document' বা রিকোয়ারমেন্ট মডেল। আর RAD-এর চূড়ান্ত ফলাফল হলো একটি কাজ করা 'Information System' বা সফটওয়্যার।

- **মানুষের সম্পৃক্ততা:** JAD-এ প্রতিষ্ঠানের অনেক স্তরের মানুষের (User, Manager, Top Management) অংশগ্রহণ থাকে। RAD-এ তুলনামূলক ছোট এবং টেকনিক্যালি দক্ষ ডেভেলপার টিম কাজ করে।

আশা করি এই গল্পের মাধ্যমে JAD এবং RAD-এর পার্থক্য এবং তাদের ব্যবহার তোমার কাছে একদম পরিষ্কার হয়ে গেছে!

সিস্টেম অ্যানালাইসিস এবং ডিজাইন (System Analysis and Design - SAD) এর জগতে তোমাকে স্বাগতম। গত ২০ বছর ধরে আমি অসংখ্য ছাত্রকে সফটওয়্যার তৈরির নানান পদ্ধতি শিখিয়েছি। সফটওয়্যার ডেভেলপমেন্টের অন্যতম দুটি গুরুত্বপূর্ণ ধারণা হলো **'Increment' (ইনক্রিমেন্ট)** এবং **'Iterative' (ইটারেটিভ)** পদ্ধতি। আজ আমি তোমাকে একটি চমৎকার এবং অর্থবহ গল্পের মাধ্যমে এই দুটি ধারণার পার্থক্য এবং ব্যবহার বিস্তারিতভাবে বোঝাব।

গল্প: একজন মাস্টার চিত্রশিল্পী ও তার দুই ক্যানভাস

ধরে নাও, তুমি আমার একজন শিক্ষানবিশ এবং আমরা সফটওয়্যারের বদলে একটি বিখ্যাত ছবি আঁকতে যাচ্ছি। আমাদের কাজ হলো মোনালিসার (Mona Lisa) মতো একটি মাস্টারপিস তৈরি করা। আমি তোমাকে শেখাচ্ছি কীভাবে দুটি ভিন্ন পদ্ধতিতে (Iterative এবং Incremental) এই একই ছবি আঁকা যায়।

প্রথম পদ্ধতি: The Iterative Approach (ইটারেটিভ পদ্ধতি) আমি প্রথমে একটি ক্যানভাস নিলাম। আমি প্রথমেই পুরো ছবিটি নিখুঁতভাবে আঁকার চেষ্টা করলাম না। বরং, আমি পেন্সিল দিয়ে পুরো ক্যানভাস জুড়ে ছবির একটি সাধারণ রূপরেখা বা স্কেচ (rough sketch) তৈরি করলাম। তুমি বুঝতে পারলে যে এটি একজন মহিলার ছবি। এরপর, দ্বিতীয় ধাপে বা **iteration**-এ আমি ছবিটিতে কিছু সাধারণ রঙ যোগ করলাম। তৃতীয় ইটারেশনে আমি চোখ, মুখের হাসি এবং পেছনের দৃশ্য আরও নিখুঁতভাবে ফুটিয়ে তুললাম।

সফটওয়্যার ডেভেলপমেন্টের ভাষায় একেই বলে **Iterative Development**। এই প্রক্রিয়ায় আমরা প্রথমেই একটি বেসিক বা সাধারণ ভার্সন তৈরি করি এবং বারবার সাইকেল (cyclic nature) বা ইটারেশনের মাধ্যমে সেটিকে পরিমার্জন (continuous refinement) করে আরও উন্নত করি। প্রতিটি ধাপে আমরা পুরো সিস্টেমটিরই কাজ করি, কিন্তু সেটি ক্রমান্বয়ে আরও নিখুঁত হতে থাকে।

দ্বিতীয় পদ্ধতি: The Incremental Approach (ইনক্রিমেন্টাল পদ্ধতি) এরপর আমি আরেকটি খালি ক্যানভাস নিলাম। এবার আমার পদ্ধতি সম্পূর্ণ ভিন্ন। আমি পুরো ছবির স্কেচ না করে, প্রথমেই শুধু মহিলার মুখমণ্ডলটি সম্পূর্ণ নিখুঁতভাবে রঙ করে শেষ করলাম। এরপর আমি তার ধড় এবং হাত সম্পূর্ণ করলাম। সর্বশেষ ধাপে আমি পেছনের দৃশ্য বা ব্যাকগ্রাউন্ডটি নিখুঁতভাবে এঁকে ছবিটি সম্পূর্ণ করলাম।

সফটওয়্যার ইঞ্জিনিয়ারিংয়ে একে বলা হয় **Incremental Development**। এই মডেলে পুরো প্রজেক্টটিকে ছোট ছোট স্বয়ংসম্পূর্ণ মডিউলে (standalone modules) ভাগ করা হয়। আমরা একটি সম্পূর্ণ অংশ বা **increment** তৈরি করে ফেলি এবং তারপর পরবর্তী অংশে হাত দিই। প্রতিটি ইনক্রিমেন্ট শেষ হওয়ার পর সেটি মূল সিস্টেমের সাথে যুক্ত (integrated) করা হয়।

বাস্তব জীবনে, বিশেষ করে **Agile Methodology**-তে আমরা এই দুটি পদ্ধতির সংমিশ্রণ ব্যবহার করি, যেখানে আমরা ছোট ছোট ইনক্রিমেন্টে সফটওয়্যার রিলিজ করি এবং ব্যবহারকারীদের ফিডব্যাক নিয়ে ইটারেটিভভাবে সেগুলোর উন্নতি সাধন করি।

Key Information in Bullet Points (গুরুত্বপূর্ণ তথ্যাবলি)

Iterative Development (ইটারেটিভ ডেভেলপমেন্ট):

- **Cyclic Process:** এটি এমন একটি প্রক্রিয়া যেখানে প্রজেক্টটি বারবার নির্দিষ্ট সাইকেল বা **iterations**-এর মাধ্যমে তৈরি হয়।
- **Continuous Refinement:** এই পদ্ধতির মূল লক্ষ্য হলো একটি সাধারণ বা প্রাথমিক ভার্সন দিয়ে শুরু করে, বারবার পরিমার্জন বা continuous refinement এর মাধ্যমে সিস্টেমটিকে উন্নত করা।
- **Evolutionary:** এটি সফটওয়্যার রিকোয়ারমেন্টের একটি ছোট অংশ দিয়ে শুরু হয় এবং সম্পূর্ণ সিস্টেম তৈরি না হওয়া পর্যন্ত ধাপে ধাপে বিকশিত (enhances the evolving versions) হতে থাকে।
- **Validation and Verification:** এই পদ্ধতির সাফল্যের চাবিকাঠি হলো প্রতিটি সাইকেলে রিকোয়ারমেন্টের কঠোর ভ্যালিডেশন এবং টেস্টিং নিশ্চিত করা।

Incremental Development (ইনক্রিমেন্টাল ডেভেলপমেন্ট):

- **Standalone Modules:** এই মডেলে পুরো প্রজেক্টটিকে একাধিক ছোট এবং স্বয়ংসম্পূর্ণ মডিউলে (standalone modules) ভাগ করে ডেভেলপ করা হয়।
- **Step-by-Step Integration:** সিস্টেমটি ছোট ছোট **increments**-এ তৈরি করা হয় এবং প্রতিটি ইনক্রিমেন্ট শেষ হওয়ার পর তা মূল সিস্টেমের সাথে যুক্ত (integrated) করা হয়।
- **Added Functionality:** প্রতিটি নতুন রিলিজ আগের রিলিজের সাথে নতুন ফাংশন যোগ করে, যতক্ষণ না ডিজাইনে থাকা সমস্ত ফাংশনালিটি সম্পূর্ণভাবে বাস্তবায়িত হয়।
- **Iterative Base:** ইনক্রিমেন্টাল ডেভেলপমেন্ট সাধারণত একটি ইটারেটিভ লাইফ সাইকেলের উপর ভিত্তি করেই পরিচালিত হয়, যেখানে একটি ইনক্রিমেন্ট তৈরি করতে এক বা একাধিক ইটারেশন লাগতে পারে।

আজ আমরা শিখব **Strategic Planning** বা কৌশলগত পরিকল্পনার একটি অত্যন্ত গুরুত্বপূর্ণ ধাপ—**'SWOT' (সোয়াট) অ্যানালাইসিস**।

চলো, এই চমৎকার ধারণাটি একটি বাস্তবমুখী এবং অর্থবহ গল্পের মাধ্যমে শিখে নিই।

গল্প: 'টেকবাংলা'র নতুন স্বপ্ন

ধরে নাও, 'টেকবাংলা' নামের একটি ছোট কিন্তু সম্ভাবনাময় আইটি (IT) ফার্ম রয়েছে। এর সিইও (CEO) হলেন শফিক। সম্প্রতি শফিক সিদ্ধান্ত নিলেন যে, তারা একটি নতুন বিটুবি (B2B) সফটওয়্যার তৈরি করবেন। কিন্তু কাজ শুরু করার আগে তিনি আমার কাছে পরামর্শ নিতে এলেন।

আমি শফিককে বললাম, "শফিক, যেকোনো দীর্ঘমেয়াদী বা কৌশলগত পরিকল্পনা (Strategic planning) শুরু করার আগে আমাদের কোম্পানির বর্তমান অবস্থা বুঝতে হবে। এর জন্য সবচেয়ে ভালো উপায় হলো একটি **SWOT analysis** করা"।

শফিক অবাক হয়ে জিজ্ঞেস করল, "SWOT কী?"

আমি একটি কাগজ নিয়ে চার ভাগে ভাগ করলাম এবং শফিককে বোঝাতে শুরু করলাম:

১. **Strengths (শক্তি):** আমি শফিককে প্রশ্ন করলাম, "আমাদের কোম্পানির সবচেয়ে বড় শক্তি কী?" শফিক হাসিমুখে বলল, "আমাদের অসাধারণ কিছু ওয়েব ডিজাইনার আছে এবং আমাদের নেটওয়ার্কও সম্প্রতি আপগ্রেড করা হয়েছে।" আমি বললাম, ঠিক তাই! এগুলো হলো আমাদের **Strengths**। আমাদের ভাবতে হবে কীভাবে এই শক্তিগুলোকে কাজে লাগিয়ে আমরা আমাদের ব্যবসায়িক লক্ষ্য অর্জন করতে পারি।

২. **Weaknesses (দুর্বলতা):** এবার আমি শফিককে বাস্তবতার মুখোমুখি করলাম। "আমাদের কিছু দুর্বলতাও আছে, তাই না?" শফিক দীর্ঘশ্বাস ফেলে বলল, "হ্যাঁ, আমরা এখনো কিছু পুরনো বা লিগ্যাসি সিস্টেম (legacy systems) ব্যবহার করছি। তাছাড়া, গত মাসে আমাদের বাজেট বাড়ানোর আবেদনও বাতিল হয়ে গেছে।" আমি বললাম, এগুলোই আমাদের **Weaknesses**। এখন আমাদের পরিকল্পনা করতে হবে কীভাবে এই দুর্বলতাগুলো কমানো বা দূর করা যায়।

৩. **Opportunities (সুযোগ):** এবার আমরা ভবিষ্যতের সম্ভাবনার দিকে তাকালাম। আমি বললাম, "বর্তমানে বাজারে বিটুবি (B2B) সফটওয়্যারের ব্যাপক চাহিদা। আমরা যদি দ্রুত এই নতুন সফটওয়্যারটি বাজারে আনতে পারি, তবে আমরাই হব প্রথম।" এই বাইরের

সম্ভাবনাগুলোই হলো আমাদের **Opportunities**। আমাদের ভাবতে হবে কীভাবে আমরা এই সুযোগগুলোর সর্বোচ্চ সদ্ব্যবহার করতে পারি।

৪. **Threats (হুমকি):** সবশেষে আমি শফিককে সতর্ক করে বললাম, "বাজারে আমাদের কিছু আক্রমণাত্মক নতুন ওয়েব প্রতিযোগী (Aggressive new Web competition) এসেছে। তাছাড়া, অন্য বড় কোম্পানিগুলো আমাদের কর্মীদের আরও ভালো সুযোগ-সুবিধা (better benefits) অফার করছে, যা আমাদের জন্য ঝুঁকির কারণ।" এগুলো হলো আমাদের **Threats**। আমাদের মূল্যায়ন করতে হবে কীভাবে আমরা এই সম্ভাব্য ঝুঁকিগুলো পরিচালনা এবং মোকাবেলা করব।

এই গল্প থেকে শফিক খুব সহজেই বুঝে গেল যে, নতুন প্রজেক্টে হাত দেওয়ার আগে তার টিমের শক্তিকে কাজে লাগাতে হবে, দুর্বলতাগুলো কাটাতে হবে, সুযোগগুলো লুফে নিতে হবে এবং বাইরের হুমকিগুলোর জন্য আগাম প্রস্তুতি নিতে হবে।

Key Information in Bullet Points (গুরুত্বপূর্ণ তথ্যাবলি)

গল্পের মাধ্যমে আমরা যা শিখলাম, তা সিস্টেম অ্যানালাইসিস এবং ডিজাইনের ভাষায় নিচে তুলে ধরা হলো:

- **SWOT Analysis:** এটি একটি ম্যানেজমেন্ট রিভিউ পদ্ধতি, যা **S**trengths (শক্তি), **W**eaknesses (দুর্বলতা), **O**pportunities (সুযোগ) এবং **T**hreats (হুমকি) এর আদ্যক্ষর নিয়ে গঠিত।
- **Foundation of Strategic Planning:** একটি সঠিক কৌশলগত পরিকল্পনা (Strategic planning) তৈরির জন্য SWOT অ্যানালাইসিস হলো একটি শক্ত ভিত্তি (solid foundation)।
- **Resource Evaluation:** এই অ্যানালাইসিস একটি প্রতিষ্ঠানের কারিগরি (technical), মানবসম্পদ (human) এবং আর্থিক (financial) বিষয়গুলো গভীরভাবে পরীক্ষা করে।
- **Core Questions Addressed:** SWOT অ্যানালাইসিস মূলত নিচের চারটি গুরুত্বপূর্ণ প্রশ্নের উত্তর খোঁজে:
    - **Strengths:** আমাদের শক্তিগুলো কী এবং ব্যবসায়িক লক্ষ্য অর্জনে আমরা সেগুলোকে কীভাবে ব্যবহার করতে পারি?
    - **Weaknesses:** আমাদের দুর্বলতাগুলো কী এবং কীভাবে আমরা সেগুলো কমাতে বা দূর করতে পারি?
    - **Opportunities:** আমাদের সামনে কী কী সুযোগ রয়েছে এবং কীভাবে আমরা সেগুলোর সদ্ব্যবহার করার পরিকল্পনা করতে পারি?

- **Threats:** আমাদের জন্য হুমকিগুলো কী এবং কীভাবে আমরা সম্ভাব্য ঝুঁকিগুলো মূল্যায়ন ও মোকাবেলা করতে পারি?

আশা করি এই গল্পের মাধ্যমে SWOT অ্যানালাইসিস-এর ধারণাটি তোমার কাছে একদম পরিষ্কার হয়ে গেছে!

গত ২০ বছর ধরে আমি বিভিন্ন প্রজেক্ট এবং ছাত্র-ছাত্রীদের সাথে কাজ করে দেখেছি যে, যেকোনো সফটওয়্যার বা সিস্টেম ডেভেলপমেন্টের শুরুতে দুটি বিষয় সবচেয়ে বেশি গুরুত্বপূর্ণ—**'Reason' (কারণ)** এবং **'Factor' (প্রভাবক)**। কেন একটি সিস্টেম তৈরি করা হচ্ছে এবং কোন প্রভাবকগুলো এর পেছনে কাজ করছে, তা বোঝা একজন অ্যানালিস্টের জন্য অত্যাবশ্যক।

চলো, এই জটিল ধারণাটি একটি বাস্তবমুখী এবং সুন্দর গল্পের মাধ্যমে শিখে নিই।

গল্প: 'রূপক এন্টারপ্রাইজ' এর নতুন সিস্টেমের খোঁজে

ধরে নাও, 'রূপক এন্টারপ্রাইজ' নামের একটি ঐতিহ্যবাহী এবং জনপ্রিয় পোশাকের ব্র্যান্ড আছে। এর মালিক রূপক একদিন আমার কাছে এসে বলল, "ম্যাডাম, আমি আমার ব্যবসার জন্য একটি নতুন আইটি (IT) সিস্টেম বানাতে চাই। আমার মনে হয় আমাদের এখন একটি আধুনিক সফটওয়্যার দরকার।"

আমি হাসিমুখে বললাম, "খুব ভালো কথা রূপক! কিন্তু কাজ শুরু করার আগে আমাকে বুঝতে হবে তোমার এই সিস্টেমটি চাওয়ার পেছনের মূল **Reason** বা কারণগুলো কী এবং কোন **Factor** বা প্রভাবকগুলো তোমাকে এই সিদ্ধান্ত নিতে বাধ্য করেছে।"

রূপক কিছুটা চিন্তায় পড়ে গেল। আমি তখন তাকে সহজ করে বোঝাতে শুরু করলাম।

Reason (কারণ): আমি বললাম, "রূপক, যেকোনো সিস্টেম প্রজেক্ট শুরু করার প্রধানত দুটি বা তিনটি বড় **Reason** থাকে। ১. **Problems (সমস্যা):** তোমার বর্তমান ব্যবসায় কি এমন কোনো সমস্যা হচ্ছে যা তুমি সমাধান করতে চাও? রূপক লাফিয়ে উঠে বলল, "হ্যাঁ! আমাদের বর্তমান ম্যানুয়াল ইনভেন্টরি সিস্টেমে অনেক ভুল হয় এবং কাজ খুব ধীরগতিতে চলে। গত মাসে আমাদের বেশ কিছু হিসাব মেলেনি।" ২. **Opportunities (সুযোগ):** আমি বললাম, "দ্বিতীয় কারণটি হলো সুযোগ। তুমি কি এমন কোনো সুযোগ দেখতে পাচ্ছ যা নতুন সিস্টেম দিয়ে কাজে লাগানো সম্ভব?" রূপক জানাল, "অবশ্যই! আমরা যদি একটি নতুন সিস্টেমের মাধ্যমে অনলাইনে বিক্রি শুরু করতে পারি, তবে আমাদের ব্যবসার প্রসার দ্রুত হবে। এটি একটি বড় সুযোগ।"

Factor (প্রভাবক): "এবার আসি **Factor**-এর কথায়," আমি বুঝিয়ে বলতে লাগলাম। "এই যে তুমি নতুন সিস্টেমের কথা ভাবছ, এর পেছনে তোমার ব্যবসার ভেতরের এবং বাইরের অনেকগুলো শক্তি বা উপাদান কাজ করছে। এগুলোকে আমরা দুটি ভাগে ভাগ করতে পারি—**Internal Factors** এবং **External Factors**।

রূপককে আমি আমার প্যাডে এঁকে দেখালাম:

- **Internal Factors (ভেতরের প্রভাবক):** "তোমার শীর্ষ কর্মকর্তারা (Top managers), যেমন তোমার বাবা, হয়তো চাইছেন ব্যবসাটাকে বড় করতে। আবার তোমার কর্মীরা (User requests) হয়তো প্রতিদিনের কাজে ম্যানুয়াল সিস্টেমের ধীরগতি নিয়ে অভিযোগ করছে এবং ভালো কোনো সেবা চাইছে। হয়তো তোমাদের বর্তমান ডেটা বা সিস্টেমে (Existing systems and data) অনেক ভুল থাকছে, যা এই পরিবর্তনের তাগিদ দিচ্ছে।"
- **External Factors (বাইরের প্রভাবক):** "এবার ভাবো বাইরের জগতের কথা। প্রযুক্তি (Technology) দ্রুত বদলাচ্ছে। তোমার গ্রাহকরা (Customers) হয়তো এখন দোকানে না এসে ঘরে বসে অনলাইনে অর্ডার করতে চায়। অন্যদিকে, তোমার প্রতিদ্বন্দ্বীরা (Competitors) এরই মধ্যে নতুন ওয়েব অ্যাপ নিয়ে বাজারে আক্রমণাত্মক ব্যবসা শুরু করেছে। এছাড়া সাপ্লায়ারদের (Suppliers) সাথে জাস্ট-ইন-টাইম (JIT) ইনভেন্টরি সিস্টেমের মাধ্যমে দ্রুত যোগাযোগের প্রয়োজন পড়েছে। এগুলো সবই হলো তোমার এক্সটার্নাল ফ্যাক্টর, যা তোমাকে বাধ্য করছে নতুন সিস্টেমের দিকে ঝুঁকতে।"

এই গল্পের পর রূপকের কাছে তার প্রজেক্টের **Reason** (সে কেন সিস্টেমটি চাইছে) এবং **Factor** (কোন ভেতরের ও বাইরের চাপগুলো তাকে এই সিদ্ধান্ত নিতে সাহায্য করছে) একেবারে দিনের আলোর মতো পরিষ্কার হয়ে গেল।

Key Information in Bullet Points (গুরুত্বপূর্ণ তথ্যাবলি)

সিস্টেম অ্যানালাইসিসের ভাষায় **Reason** এবং **Factor** এর মূল ধারণাগুলো নিচে দেওয়া হলো:

Reasons for Systems Projects (সিস্টেম প্রজেক্ট শুরু করার কারণসমূহ):

- **Solving Problems (সমস্যা সমাধান):** বর্তমান সিস্টেমে যখন অতিরিক্ত ভুল হয়, কাজ অসম্পূর্ণ থাকে বা ধীরগতিতে চলে, তখন নতুন সিস্টেমের প্রয়োজন হয়।
- **Seizing Opportunities (সুযোগের সদ্ব্যবহার):** নতুন সিস্টেম ব্যবহার করে প্রতিষ্ঠানের উন্নতি করা, সেবার মান বাড়ানো বা প্রতিযোগিতামূলক সুবিধা (competitive advantage) অর্জনের সুযোগ থাকলে প্রজেক্ট শুরু করা হয়।
- **Achieving Objectives (লক্ষ্য অর্জন):** প্রাতিষ্ঠানিক বা ব্যবসায়িক লক্ষ্য অর্জনের উদ্দেশ্যে সিস্টেম প্রজেক্ট হাতে নেওয়া হয়।

Factors Affecting System Projects (সিস্টেম প্রজেক্টকে প্রভাবিত করা ফ্যাক্টরসমূহ):
সিস্টেম প্রজেক্ট মূলত দু'ধরনের ফ্যাক্টর দ্বারা প্রভাবিত হয়:

১. Internal Factors (অভ্যন্তরীণ প্রভাবকসমূহ):

- **Strategic Plan:** কোম্পানির কৌশলগত পরিকল্পনা বা দীর্ঘমেয়াদী লক্ষ্য আইটি (IT) প্রজেক্টকে দিকনির্দেশনা দেয়।
- **Top Managers:** শীর্ষ ব্যবস্থাপনা বা ডিরেক্টরদের নির্দেশনায় বড় প্রজেক্টগুলো শুরু হয়।
- **User Requests:** ব্যবহারকারী বা কর্মীদের কাজের সুবিধার জন্য নতুন সিস্টেম বা উন্নতির চাহিদাও প্রজেক্টের জন্ম দেয়।
- **Information Technology Department:** আইটি বিভাগ অনেক সময় নতুন সিস্টেম বা আপগ্রেড করার প্রস্তাব দেয়।
- **Existing Systems and Data:** বর্তমান সিস্টেমের ত্রুটি বা ডেটার সীমাবদ্ধতা নতুন প্রজেক্ট শুরু করতে বাধ্য করে।

২. External Factors (বাহ্যিক প্রভাবকসমূহ):

- **Technology:** প্রযুক্তির দ্রুত পরিবর্তন নতুন সিস্টেম গ্রহণ করতে বাধ্য করে।
- **Customers:** গ্রাহকদের সেবার মান উন্নত করতে এবং তাদের চাহিদা মেটাতে (যেমন- CRM সিস্টেম) নতুন প্রজেক্ট নেওয়া হয়।
- **Competitors:** বাজারে টিকে থাকতে এবং প্রতিযোগীদের মোকাবিলা করতে নতুন সেবার প্রবর্তন করতে হয়।
- **Suppliers:** সাপ্লায়ারদের সাথে নিরবচ্ছিন্ন যোগাযোগের জন্য (যেমন- EDI বা JIT সিস্টেম) সিস্টেম আপগ্রেড করতে হয়।
- **The Economy & Government:** দেশের অর্থনৈতিক অবস্থা এবং সরকারি আইন বা রেগুলেশন (Regulations) সিস্টেমের ডিজাইনে বড় প্রভাব ফেলে।

গত ক্লাসগুলোতে আমরা 'Increment and Iterative', 'SWOT' এবং 'Reason and Factor' নিয়ে আলোচনা করেছি। আজ আমরা শিখব সিস্টেম ডেভেলপমেন্টের অন্যতম বড় একটি স্তম্ভ—**Project Management** বা প্রজেক্ট ম্যানেজমেন্টের ৪টি মূল ধাপ (**4 Basic Phases of Project Management**) সম্পর্কে।

চলো, বরাবরের মতোই একটি চমৎকার গল্পের মাধ্যমে এই ৪টি ধাপ বিস্তারিতভাবে শিখে নিই।

গল্প: 'ভোজন বিলাস'-এর নতুন অ্যাপ তৈরির রূপকথা

ধরে নাও, 'ভোজন বিলাস' শহরের খুব জনপ্রিয় একটি রেস্টুরেন্ট। এর মালিক আনিস সাহেব একদিন আমার কাছে এলেন। তিনি বললেন, "ম্যাডাম, আমার রেস্টুরেন্টের জন্য একটি দারুণ হোম-ডেলিভারি অ্যাপ বানাতে চাই, কিন্তু আমি জানি না কোথা থেকে শুরু করব।"

আমি হাসিমুখে বললাম, "আনিস সাহেব, যেকোনো সিস্টেম বা সফটওয়্যার তৈরি করতে গেলে আমাদের **Project Management**-এর ৪টি নির্দিষ্ট ধাপ পার হতে হয়। এগুলো হলো: **Initiating**, **Planning**, **Executing**, এবং **Closing down**। আসুন, আপনার অ্যাপ বানানোর প্রক্রিয়ার মাধ্যমেই ধাপগুলো বুঝিয়ে বলি।"

প্রথম ধাপ: Initiating the project (প্রজেক্ট শুরু করা) আমি আনিস সাহেবকে বললাম, "প্রথমেই আমরা সরাসরি কোডিং শুরু করব না। প্রথমে আমাদের প্রজেক্টটি শুরু করার পরিবেশ তৈরি করতে হবে।" আমরা প্রথমে একটি টিম গঠন করলাম (Establishing the Project Initiation Team)। এরপর প্রজেক্টের আকার এবং জটিলতা পরিমাপ করলাম। আমি একটি **Project Workbook** তৈরি করলাম, যেখানে আমাদের সমস্ত আলোচনা, রিপোর্ট এবং কাজের হিসাব রেকর্ড করা হবে। সবশেষে আমরা একটি **Project Charter** তৈরি করলাম, যা হলো একটি সংক্ষিপ্ত ডকুমেন্ট—যেখানে লেখা থাকবে এই অ্যাপটি কী কী সুবিধা দেবে, কবে শুরু হবে এবং কবে শেষ হবে। এটি দেখে আনিস সাহেব খুব খুশি হলেন কারণ তিনি স্পষ্ট বুঝতে পারলেন প্রজেক্টের লক্ষ্য কী।

দ্বিতীয় ধাপ: Planning the project (প্রজেক্টের পরিকল্পনা) এরপর আমরা দ্বিতীয় ধাপে গেলাম। আমি বললাম, "এবার আমাদের বিস্তারিত পরিকল্পনা করতে হবে।" আমরা প্রথমে প্রজেক্টের পরিধি বা স্কোপ (Describing project scope) নির্ধারণ করলাম এবং এটি আসলেই বাস্তবে করা সম্ভব কি না তা যাচাই করলাম। এরপর পুরো অ্যাপ তৈরির কাজটিকে ছোট ছোট অংশে (manageable tasks) ভাগ করে ফেললাম। কতজন লোক লাগবে, কত বাজেট লাগবে (preliminary budget) এবং কাজ শেষ করতে কতদিন সময় লাগবে (preliminary schedule), তার একটি রূপরেখা তৈরি করলাম। সবশেষে, একটি মূল প্ল্যান তৈরি করলাম যাকে বলা হয় **Baseline project plan**, যা আমাদের পরবর্তী কাজগুলোকে গাইড করবে।

তৃতীয় ধাপ: Executing the project (প্রজেক্ট বাস্তবায়ন) এবার আমাদের আসল কাজ শুরু হলো। আমাদের টিম কোডিং এবং ডিজাইন শুরু করল। এই ধাপে আমাদের মূল কাজ হলো আগের ধাপে তৈরি করা বেসলাইন প্ল্যানটি বাস্তবে রূপ দেওয়া (Executing the baseline project plan)। কাজ চলার সময় আমি খেয়াল রাখছিলাম টিম ঠিকমতো এগোচ্ছে কি না (Monitoring project progress)। মাঝখানে আনিস সাহেব এসে বললেন, "অ্যাপে বিকাশ পেমেন্টের পাশাপাশি ক্রেডিট কার্ড পেমেন্টও যোগ করা যায়?" আমি তখন প্ল্যানে সেই অনুযায়ী পরিবর্তন আনলাম (Managing changes to the baseline project plan)। এই পুরো সময় জুড়ে আমি আনিস সাহেব এবং টিমের সদস্যদের সাথে কাজের আপডেট নিয়ে নিয়মিত যোগাযোগ (Communicating the project status) রাখছিলাম।

চতুর্থ ধাপ: Closing down the project (প্রজেক্ট সমাপ্তকরণ) অবশেষে আমাদের অ্যাপ তৈরি শেষ! আমরা শেষ ধাপে পৌঁছে গেলাম। এখানে আমাদের কাজ হলো প্রজেক্টটি আনুষ্ঠানিকভাবে শেষ করা (Closing down the project)। আমরা দেখলাম যে অ্যাপটি ঠিকঠাক কাজ করছে এবং আনিস সাহেবও খুব সন্তুষ্ট। আমরা আমাদের টিমের কাজের মূল্যায়ন করলাম এবং ভবিষ্যতে আরও ভালো কীভাবে কাজ করা যায় তা নিয়ে আলোচনা করলাম (Conducting postproject reviews)। সবশেষে, আনিস সাহেবের সাথে আমাদের সমস্ত চুক্তি এবং শর্ত পূরণ হয়েছে কি না তা নিশ্চিত করে প্রজেক্টটি আনুষ্ঠানিকভাবে হস্তান্তর করলাম (Closing the customer contract)।

এই গল্পের মাধ্যমে আনিস সাহেব খুব সহজেই বুঝে গেলেন যে একটি প্রজেক্ট কীভাবে সফলভাবে পরিচালনা করতে হয়।

Key Information in Bullet Points (গুরুত্বপূর্ণ তথ্যাবলি)

প্রজেক্ট ম্যানেজমেন্ট হলো কোনো নির্দিষ্ট লক্ষ্য অর্জনের জন্য একটি টিমকে পরিচালনা করা, পরিকল্পনা করা এবং কাজ নিয়ন্ত্রণ করার প্রক্রিয়া। এর প্রধান ৪টি ধাপ নিচে দেওয়া হলো:

1. Initiating the project (প্রজেক্টের সূচনা):

- **Team Formation:** প্রজেক্টের কাজগুলো সম্পন্ন করার জন্য একটি প্রাথমিক কোর টিম গঠন করা (Establishing the Project Initiation Team)।
- **Project Workbook:** একটি অনলাইন বা হার্ড-কপি রিপোজিটরি তৈরি করা, যেখানে প্রজেক্টের সমস্ত চিঠিপত্র, ইনপুট, আউটপুট এবং রিপোর্ট সংরক্ষিত থাকে।

- **Project Charter:** প্রজেক্টের লক্ষ্য, ডেলিভারেবলস, ম্যানেজারের নাম, এবং শুরুর ও শেষের তারিখ উল্লেখ করে একটি সংক্ষিপ্ত (সাধারণত এক পৃষ্ঠার) হাই-লেভেল ডকুমেন্ট তৈরি করা।

2. Planning the project (প্রজেক্টের পরিকল্পনা):

- **Scope and Feasibility:** প্রজেক্টের স্কোপ এবং বিকল্প সমাধানগুলোর ফিজিবিলিটি (feasibility) সংজ্ঞায়িত করা।
- **Manageable Tasks:** পুরো প্রজেক্টটিকে ছোট এবং পরিচালনাযোগ্য কাজ বা টাস্কে ভাগ করা।
- **Resource and Schedule:** প্রয়োজনীয় রিসোর্সের অনুমান করা, বাজেট তৈরি করা এবং একটি প্রাথমিক শিডিউল বা সময়সূচি (preliminary schedule) নির্ধারণ করা।
- **Baseline Project Plan:** সবকিছু মিলিয়ে একটি বেসলাইন প্ল্যান তৈরি করা, যা প্রজেক্টের পরবর্তী ধাপগুলোকে গাইড করবে।

3. Executing the project (প্রজেক্টের বাস্তবায়ন):

- **Action:** বেসলাইন প্ল্যানটিকে কাজে পরিণত করা অর্থাৎ প্রজেক্টের আসল কাজ (Executing the baseline plan) শুরু করা।
- **Monitoring Progress:** বেসলাইন প্ল্যানের সাপেক্ষে প্রজেক্টের অগ্রগতি পর্যবেক্ষণ করা এবং শিডিউল বা বাজেট ঠিক রাখা।
- **Managing Changes:** কাজের মাঝে কোনো নতুন রিকোয়ারমেন্ট আসলে সে অনুযায়ী পরিবর্তনগুলো ম্যানেজ করা এবং ওয়ার্কবুকে আপডেট রাখা।
- **Communication:** প্রজেক্টের বর্তমান স্ট্যাটাস বা অগ্রগতি সম্পর্কে টিম মেম্বার, ম্যানেজার এবং কাস্টমারদের নিয়মিত জানানো।

4. Closing down the project (প্রজেক্ট সমাপ্তকরণ):

- **Project Termination:** প্রজেক্টটি সফলভাবে সম্পন্ন হলে এর প্রাকৃতিক সমাপ্তি (natural termination) ঘোষণা করা।
- **Postproject Reviews:** প্রজেক্ট শেষ হওয়ার পর এর শক্তিশালী ও দুর্বল দিকগুলো চিহ্নিত করা এবং ম্যানেজমেন্ট ও কাস্টমারদের সাথে চূড়ান্ত রিভিউ সম্পন্ন করা।
- **Closing Customer Contract:** কাস্টমারের সাথে সব চুক্তিভিত্তিক বাধ্যবাধকতা (contractual obligations) পূরণ হয়েছে কি না তা নিশ্চিত করা।

আশা করি এই গল্প এবং পয়েন্টগুলো প্রজেক্ট ম্যানেজমেন্টের ৪টি ধাপ সম্পর্কে তোমার ধারণাকে একদম স্বচ্ছ করে দিয়েছে!

আজ আমরা শিখব যেকোনো প্রজেক্টের সবচেয়ে গুরুত্বপূর্ণ অর্থনৈতিক দিক—**'Cost/Benefit Analysis' (কস্ট-বেনিফিট অ্যানালাইসিস)**।

চলো, এই চমৎকার এবং হিসাব-নিকাশের ধারণাটি একটি বাস্তব ও অর্থবহ গল্পের মাধ্যমে শিখে নিই।

গল্প: সাজিদের স্বপ্নের 'শেবা কুরিয়ার' এবং একটি বড় সিদ্ধান্ত

সাজিদ শহরের একটি উঠতি কুরিয়ার সার্ভিস 'শেবা কুরিয়ার'-এর মালিক। তার ব্যবসা বেশ বড় হচ্ছে, কিন্তু সব হিসাব-নিকাশ, পার্সেল ট্র্যাকিং এবং কর্মীদের হাজিরা সে এখনো খাতায়-কলমেই (manual system) রাখে। একদিন সে আমার কাছে এসে বলল, "ম্যাডাম, আমি ব্যবসার জন্য একটি আধুনিক সফটওয়্যার সিস্টেম বানাতে চাই। এতে আমার প্রায় ৫ লাখ টাকা খরচ হবে। কিন্তু আমি বুঝতে পারছি না, এই বিশাল অঙ্কের টাকা বিনয়োগ করা আমার জন্য ঠিক হবে কি না?"

আমি হাসিমুখে বললাম, "সাজিদ, তোমার এই প্রশ্নের উত্তর দিতে হলে আমাদের **Economic Feasibility** বা অর্থনৈতিক সম্ভাব্যতা যাচাই করতে হবে। আর এর জন্যই আমরা করব **'Cost/Benefit Analysis'**। চলো দেখি তোমার এই নতুন সিস্টেমে কী কী খরচ (Costs) আছে এবং কী কী সুবিধা (Benefits) আছে।"

আমি একটি সাদা বোর্ড নিয়ে সাজিদের প্রজেক্টের হিসাব শুরু করলাম।

ধাপ ১: খরচ এবং সুবিধা চিহ্নিত করা (Identify Costs and Benefits) আমি সাজিদকে বললাম, "সিস্টেমের খরচ এবং সুবিধাকে আমরা মূলত দুই ভাগে ভাগ করতে পারি—**Tangible (দৃশ্যমান)** এবং **Intangible (অদৃশ্য বা অস্পর্শনীয়)**।"

- **Tangible Costs (দৃশ্যমান খরচ):** যে খরচগুলো আমরা সরাসরি টাকায় মাপতে পারি। যেমন- নতুন কম্পিউটার কেনা, সফটওয়্যার ডেভেলপারদের বেতন, কর্মীদের ট্রেনিংয়ের খরচ ইত্যাদি। এর মধ্যে কিছু খরচ আছে **One-Time Costs** (এককালীন খরচ), যেমন সফটওয়্যারটি তৈরি করার খরচ। আবার কিছু আছে **Recurring Costs** বা **Operational Costs** (চলমান খরচ), যেমন প্রতি মাসের ইন্টারনেট বিল, মেইনটেন্যান্স বা বিদ্যুৎ বিল।
- **Intangible Costs (অদৃশ্য খরচ):** যেগুলো টাকায় মাপা যায় না। যেমন, নতুন সিস্টেমটি যখন প্রথম চালু হবে, তখন কর্মীরা কাজ বুঝতে গিয়ে কিছু ভুল করতে পারে, যার ফলে সাময়িকভাবে কিছু কাস্টমার অসন্তুষ্ট হতে পারে বা কোম্পানির সুনাম কিছুটা ক্ষুণ্ন হতে পারে।

এবার আসি সুবিধার (Benefits) কথায়:

- **Tangible Benefits (দৃশ্যমান সুবিধা):** যে সুবিধাগুলো সরাসরি টাকায় হিসাব করা যায়। যেমন– সফটওয়্যার ব্যবহারের ফলে কাজের গতি বাড়বে, প্রতিদিন আরও বেশি পার্সেল ডেলিভারি দেওয়া যাবে। খাতা-কলমের খরচ বাঁচবে এবং অতিরিক্ত কর্মী নিয়োগ দেওয়ার প্রয়োজন হবে না (reduction in staff)।
- **Intangible Benefits (অদৃশ্য সুবিধা):** যে সুবিধাগুলো টাকায় মাপা কঠিন কিন্তু ব্যবসার জন্য অমূল্য। যেমন– কাস্টমাররা ঘরে বসে পার্সেল ট্র্যাক করতে পেরে অনেক খুশি হবে (improved customer service), কর্মীদের কাজের একঘেয়েমি কমবে (employee morale) এবং তোমার দ্রুত সিদ্ধান্ত নেওয়ার ক্ষমতা বাড়বে (improved decision-making)।

ধাপ ২ ও ৩: টাকার অঙ্ক বসানো এবং ক্যাশ ফ্লো নির্ধারণ (Assign Values and Determine Cash Flow) সাজিদ ও আমি মিলে অনুমান করে দেখলাম যে, প্রথম বছরে খরচ বেশি হলেও, দ্বিতীয় বছর থেকে খরচ কমে যাবে এবং প্রতি বছর তার লাভ বা আয় (cash inflow) বাড়তে থাকবে।

ধাপ ৪: অর্থনৈতিক মান যাচাই করা (Assess Project's Economic Value) সবশেষে আমি সাজিদকে বললাম, "এই বিনিয়োগটি যৌক্তিক কি না, তা আমরা তিনটি মেট্রিক বা পদ্ধতির মাধ্যমে যাচাই করব: ১. **Return on Investment (ROI):** আমরা হিসাব করে দেখলাম, মোট খরচের তুলনায় তোমার লাভের হার কত। যদি ROI বেশি হয়, তবে প্রজেক্টটি লাভজনক। ২. **Break-Even Point (BEP):** যে পয়েন্টে বা সময়ে তোমার খরচ করা ৫ লাখ টাকা পুরোপুরি উঠে আসবে এবং তুমি লাভের মুখ দেখতে শুরু করবে। সাজিদের ক্ষেত্রে দেখলাম, আড়াই বছর পর তার প্রজেক্ট Break-Even পয়েন্টে পৌঁছাবে। ৩. **Net Present Value (NPV) এবং Time Value of Money:** আজকে তোমার কাছে যে ৫ লাখ টাকা আছে, ভবিষ্যতে তার মান কমে যাবে (মুদ্রাস্ফীতির কারণে)। তাই ভবিষ্যতের লাভের টাকাকে আজকের মূল্যে (present value) রূপান্তর করে দেখলাম যে, তোমার প্রজেক্টের NPV পজিটিভ (Positive) আসছে।

হিসাব শেষে সাজিদ স্বস্তির নিশ্বাস ফেলল। সে বুঝতে পারল, শুরুতে ৫ লাখ টাকা খরচ হলেও, দীর্ঘমেয়াদে এই সিস্টেম তাকে বিশাল মুনাফা ও সুনাম এনে দেবে।

Key Information in Bullet Points (গুরুত্বপূর্ণ তথ্যাবলি)

Cost/Benefit Analysis-এর মূল ধারণাগুলো নিচে দেওয়া হলো:

- **Cost-Benefit Analysis কী?** এটি Economic Feasibility-এর একটি অংশ, যেখানে কোনো প্রজেক্টের খরচ এবং সেখান থেকে প্রাপ্ত সুবিধার তুলনা করে প্রজেক্টটির অর্থনৈতিক গ্রহণযোগ্যতা যাচাই করা হয়।
- **Steps of Economic Feasibility (৪টি ধাপ):**
    1. Identify Costs and Benefits (খরচ ও সুবিধাগুলো চিহ্নিত করা)।
    2. Assign Values to Costs and Benefits (সম্ভাব্য খরচের পরিমাণ বা টাকার অঙ্ক নির্ধারণ করা)।
    3. Determine Cash Flow (ক্যাশ ফ্লো বা নগদ অর্থের প্রবাহ বের করা)।
    4. Assess Project's Economic Value (প্রজেক্টের অর্থনৈতিক মান যাচাই করা)।
- **Categories of Costs & Benefits:**
    - **Tangible Costs/Benefits:** যেগুলো সরাসরি টাকায় পরিমাপ করা যায় (যেমন- হার্ডওয়্যার খরচ, লোকবল কমানোর ফলে বাঁচানো টাকা)।
    - **Intangible Costs/Benefits:** যেগুলো টাকায় পরিমাপ করা যায় না, কিন্তু ব্যবসার ওপর প্রভাব ফেলে (যেমন- কাস্টমার সন্তুষ্টি, সুনাম হারানো)।
- **Types of Costs:**
    - **One-time Costs:** প্রজেক্ট শুরু ও ডেভেলপমেন্টের এককালীন খরচ।
    - **Recurring / Operational Costs:** সিস্টেম চালু রাখার জন্য নিয়মিত খরচ (যেমন- মেইনটেন্যান্স, সাপ্লাই)।
    - **Fixed vs Variable Costs:** ফিক্সড কস্ট (যেমন- লিজ বা ভাড়া) সবসময় নির্দিষ্ট থাকে, আর ভ্যারিয়েবল কস্ট (যেমন- সাপ্লাই বা ব্যবহার) সিস্টেমের ব্যবহারের ওপর নির্ভর করে বাড়ে বা কমে।
- **Evaluation Methods (মূল্যায়ন পদ্ধতিসমূহ):**
    - **Return on Investment (ROI):** মোট খরচের বিপরীতে মোট লাভের অনুপাত (Net benefits / total costs)। এটি যত বেশি, প্রজেক্ট তত লাভজনক।
    - **Break-Even Point (BEP):** প্রজেক্টের শুরুতে করা বিনিয়োগ বা খরচ যে সময়ে গিয়ে উঠে আসে (Costs = Benefits) এবং প্রজেক্টটি লাভজনক হতে শুরু করে।
    - **Net Present Value (NPV):** 'Time Value of Money' অর্থাৎ অর্থের সময়ের মূল্যের ওপর ভিত্তি করে ভবিষ্যতের সম্ভাব্য আয় ও ব্যয়ের পার্থক্য আজকের মূল্যে হিসাব করা। NPV পজিটিভ হলে প্রজেক্টটি গ্রহণযোগ্য।

গত ২০ বছর ধরে আমি বিভিন্ন প্রজেক্ট এবং শিক্ষার্থীদের সাথে কাজ করে দেখেছি, একটি প্রজেক্ট প্রযুক্তিগতভাবে যতই চমৎকার হোক না কেন, সেটি আর্থিকভাবে লাভজনক কি না, তা যাচাই করা সবচেয়ে বেশি জরুরি। এই যাচাই করার প্রক্রিয়াকেই বলা হয় **Economic Feasibility Analysis**।

চলো, এই চমৎকার এবং হিসাব-নিকাশের ধারণাটি একটি বাস্তব ও অর্থবহ গল্পের মাধ্যমে শিখে নিই, যেখানে আমরা শিখব **Steps to Conduct an Economic Feasibility Analysis** বা অর্থনৈতিক সম্ভাব্যতা যাচাই করার ধাপগুলো কী কী।

গল্প: 'নদী সুপারমার্কেট' এর নতুন সফটওয়্যারের হিসাব-নিকাশ

রফিক সাহেব শহরের একটি বড় মুদি দোকান বা 'নদী সুপারমার্কেট' এর মালিক। ব্যবসার পরিধি বাড়ার সাথে সাথে তার পুরোনো ম্যানুয়াল সিস্টেমে আর কাজ চলছে না। তাই তিনি সিদ্ধান্ত নিলেন একটি আধুনিক ইনভেন্টরি ও বিলিং সিস্টেম তৈরি করবেন। কিন্তু তিনি চিন্তায় পড়ে গেলেন, "নতুন সিস্টেমে তো অনেক টাকা খরচ হবে! এই প্রজেক্টটি করা কি আমার জন্য আর্থিকভাবে লাভজনক হবে?"

তিনি আমার কাছে এলেন পরামর্শের জন্য। আমি বললাম, "রফিক সাহেব, ভয় পাবেন না। আমরা এখন একটি **Economic Feasibility Analysis** করব। এর জন্য আমাদের ৪টি নির্দিষ্ট ধাপ (Steps) পার হতে হবে।"

ধাপ ১: Identify Costs and Benefits (খরচ এবং সুবিধা চিহ্নিত করা) আমি রফিক সাহেবকে বললাম, "প্রথমেই আমাদের খুঁজে বের করতে হবে এই নতুন সিস্টেমে আপনার কী কী খরচ হবে এবং আপনি কী কী সুবিধা পাবেন"। আমি তাকে বুঝিয়ে বললাম যে খরচ মূলত দুই ধরনের হয়: **Development costs** বা প্রজেক্ট তৈরির খরচ (যেমন- ডেভেলপারদের বেতন, নতুন হার্ডওয়্যার ও সফটওয়্যার কেনা, কর্মীদের ট্রেনিং) এবং **Operational costs** বা সিস্টেমটি চালানোর চলমান খরচ (যেমন- সফটওয়্যার লাইসেন্সিং, রক্ষণাবেক্ষণ বা মেইনটেন্যান্স)। একইভাবে সুবিধাগুলোও দুই ধরনের: **Tangible benefits** (দৃশ্যমান সুবিধা, যেমন- বিক্রি বৃদ্ধি, লোকবল কমানোর ফলে বাঁচানো টাকা) এবং **Intangible benefits** (অদৃশ্য সুবিধা, যেমন- কাস্টমারদের উন্নত সেবা, বাজারে সুনাম বৃদ্ধি)।

ধাপ ২: Assign Values to Costs and Benefits (খরচ এবং সুবিধার আর্থিক মূল্য নির্ধারণ করা) দ্বিতীয় ধাপে আমি রফিক সাহেবকে বললাম, "এবার আমাদের এই চিহ্নিত করা খরচ এবং সুবিধাগুলোকে টাকায় রূপান্তর করতে হবে বা **Assign Values** করতে হবে"। রফিক সাহেব বললেন, "কিন্তু অদৃশ্য সুবিধা বা ভবিষ্যতের খরচের সঠিক টাকার অঙ্ক আমি কীভাবে বুঝব?" আমি হাসিমুখে বললাম, "এটা একটু কঠিন মনে হতে পারে, তবে আমরা আপনার ব্যবসার ম্যানেজার এবং আইটি এক্সপার্টদের সাথে কথা বলে একটি সম্ভাব্য টাকার

অঙ্ক (dollar values) বের করব। প্রয়োজনে আমরা সম্ভাব্যতার (probability/likelihood) উপর ভিত্তি করে একটি অনুমিত হিসাব তৈরি করব"।

ধাপ ৩: Determine Cash Flow (নগদ অর্থের প্রবাহ নির্ধারণ করা) সব হিসাব পাওয়ার পর আমি একটি স্প্রেডশিট বা টেবিল খুললাম। "এবার আমরা **Determine Cash Flow** করব। এর মানে হলো, আগামী ৩ থেকে ৫ বছরের জন্য আমাদের প্রজেক্টের মোট আয় এবং ব্যয়ের একটি চিত্র তৈরি করা"। আমরা দেখলাম, প্রথম বছরে বা Year 0 তে রফিক সাহেবের প্রচুর Development cost আছে, কিন্তু কোনো লাভ নেই। তবে Year 1, Year 2 এবং Year 3 তে তার Operational cost এর চেয়ে Tangible benefits অনেক বেশি। প্রতি বছর মোট সুবিধা থেকে মোট খরচ বাদ দিয়ে আমরা **Net Cash Flow** বা নিট নগদ প্রবাহ বের করলাম।

ধাপ ৪: Assess Project's Economic Value (প্রজেক্টের অর্থনৈতিক মান যাচাই করা) সবশেষে আমি রফিক সাহেবকে বললাম, "এবার চূড়ান্ত সিদ্ধান্ত নেওয়ার পালা। আমরা এখন **Assess Project's Economic Value** করব, অর্থাৎ দেখব আপনার বিনিয়োগ করা টাকা আসলেই কতটা লাভজনক"। আমি তাকে ৩টি জাদুকরী সূত্র দেখালাম: ১. **Return on Investment (ROI):** মোট খরচের বিপরীতে আপনি কত শতাংশ লাভ করছেন (Total Revenue - Total Cost / Total Cost)। রফিক সাহেবের প্রজেক্টের ROI এলো বেশ ভালো একটি পজিটিভ পার্সেন্টেজ। ২. **Break-Even Point (BEP):** যে সময়ে গিয়ে আপনার বিনিয়োগ করা পুরো টাকা উঠে আসবে এবং আপনি লাভের মুখ দেখবেন। দেখলাম, মাত্র ২ বছর ৮ মাসের মাথায় তার খরচ উঠে আসবে। ৩. **Net Present Value (NPV):** ভবিষ্যতের লাভের টাকাকে আজকের মূল্যে (present value) রূপান্তর করে দেখলাম, তার প্রজেক্টের NPV শূন্যের চেয়ে বেশি (positive)।

রফিক সাহেবের মুখে তখন চওড়া হাসি! তিনি বুঝতে পারলেন যে, এই ৪টি ধাপের মাধ্যমে প্রমাণিত হয়েছে তার নতুন সিস্টেমটি আর্থিকভাবে সম্পূর্ণ লাভজনক বা Economically feasible।

Key Information in Bullet Points (গুরুত্বপূর্ণ তথ্যাবলি)

গল্পের মাধ্যমে আমরা যা শিখলাম, তা সিস্টেম অ্যানালাইসিস এবং ডিজাইনের ভাষায় **Steps to Conduct an Economic Feasibility Analysis** হিসেবে নিচে তুলে ধরা হলো:

- **Step 1: Identify Costs and Benefits (খরচ এবং সুবিধা চিহ্নিত করা)**

- **Development Costs:** প্রজেক্ট শুরু ও ডেভেলপমেন্টের এককালীন খরচ (One-Time Costs), যেমন- ডেভেলপমেন্ট টিমের বেতন, হার্ডওয়্যার/সফটওয়্যার ক্রয়, এবং ইউজার ট্রেনিং।
- **Operational Costs:** সিস্টেম চালু রাখার জন্য নিয়মিত বা চলমান খরচ (Recurring Costs), যেমন- সফটওয়্যার লাইসেন্সিং, হার্ডওয়্যার মেরামত এবং যোগাযোগ খরচ।
- **Tangible Benefits:** যে সুবিধাগুলো সরাসরি টাকায় পরিমাপ করা যায় (Hard savings), যেমন- বিক্রি বৃদ্ধি (Increased sales), ইনভেন্টরি বা স্টাফ খরচ কমানো।
- **Intangible Benefits:** যেগুলো টাকায় সরাসরি পরিমাপ করা যায় না, কিন্তু ব্যবসার জন্য অত্যন্ত জরুরি, যেমন- উন্নত কাস্টমার সার্ভিস বা মার্কেট শেয়ার ও ব্র্যান্ড রিকগনিশন বৃদ্ধি।

- **Step 2: Assign Values to Costs and Benefits (মূল্য নির্ধারণ করা)**
  - চিহ্নিত করা প্রতিটি খরচ এবং সুবিধার জন্য সুনির্দিষ্ট আর্থিক মূল্য বা Specific dollar values নির্ধারণ করতে হবে।
  - ভবিষ্যতের হিসাব অনুমান করার জন্য আইটি প্রফেশনাল এবং বিজনেস ইউজারদের বাস্তব অভিজ্ঞতার উপর নির্ভর করতে হয়।
  - প্রয়োজনে অনুমিত মূল্য বা সম্ভাবনা (Probability/Likelihood) ব্যবহার করে সম্ভাব্য মান (Expected value) বের করা হয়।
- **Step 3: Determine Cash Flow (ক্যাশ ফ্লো নির্ধারণ করা)**
  - ভবিষ্যতের ৩ থেকে ৫ বছরের জন্য প্রজেক্টের নগদ অর্থের আগমণ এবং নির্গমণ (Inflows and Outflows) নিয়ে একটি Cash flow projection তৈরি করা হয়।
  - প্রতি বছরের মোট সুবিধা (Total Benefits) থেকে মোট খরচ (Total Costs) বাদ দিয়ে নিট ক্যাশ ফ্লো (Net Cash Flow) বের করা হয়।
- **Step 4: Assess Project's Economic Value (অর্থনৈতিক মান যাচাই করা)**
  - **Return on Investment (ROI):** নিট লাভকে মোট খরচ দিয়ে ভাগ করে প্রজেক্টের লাভের হার বা মুনাফা অর্জন যাচাই করা হয়।
  - **Break-Even Point (BEP):** প্রজেক্টের শুরুতে করা বিনিয়োগ বা আসল টাকা কত বছরে ফেরত আসবে (Recover its original investment) তা নির্ণয় করা হয়।
  - **Net Present Value (NPV):** 'Time Value of Money' বা অর্থের সময়ের মূল্যের উপর ভিত্তি করে ভবিষ্যতের সম্ভাব্য আয় ও ব্যয়ের পার্থক্য আজকের মূল্যে হিসাব করা হয়। NPV শূন্যের চেয়ে বড় (positive) হলে প্রজেক্টটি আর্থিকভাবে গ্রহণযোগ্য বলে বিবেচিত হয়।

আশা করি এই গল্প এবং পয়েন্টগুলোর মাধ্যমে ইকোনমিক ফিজিবিলিটি অ্যানালাইসিসের ধাপগুলো তোমার কাছে একদম পরিষ্কার হয়ে গেছে!

Add a **diagram** to each answer. If one isn’t available in your textbook or online, **create it using AI**. This will improve clarity and help you **score higher marks**, Insha’Allah.